

队伍编号	MCB2403007
赛道	B

基于大数据分析的电商货量预测与仓库分配优化研究

摘 要

随着电商行业的发展,货量预测与品类分仓规划问题日益成为供应链管理中的重要研究课题。本文针对电商中的货量预测问题,基于大数据分析和机器学习思想,通过确定销售趋势、市场需求和季节性变化等指标,以提高预测精度和优化库存管理为目标,建立了相应的预测模型,并使用先进的算法对模型进行求解。

针对问题一,本研究针对库存量和销量预测问题,采用 **SARIMA 模型**和**随机森林回归模型**进行分析和建模。在库存量预测中,利用 SARIMA 模型通过数据预处理和参数优化,有效捕捉了**数据的季节性和趋势**。ADF 检验结果显示中 $p\text{-value}$ 远高于 0.05,表明数据可能具有趋势或季节性波动,适配该模型的预测模式。销量预测则使用**随机森林回归模型**,通过构建特征和处理历史数据,展示出良好的鲁棒性和准确性。随机森林回归模型预测的日销量结果的 MAE, MSE 和 RMSE 绝大部分均为 0, , R^2 为 1。这表明模型在这些品类上的拟合基本完全正确。两种模型的应用为库存管理和分仓规划提供了可靠的数据支持。

针对问题二,在第二问的仓库分配模型中,我们构建了一个**线性规划模型**,旨在通过最小化仓租成本来优化各品类的仓库分配,特别强调“一品一仓”原则,以确保每个品类仅分配至一个仓库,从而简化管理流程并提高效率。同时,该模型也满足仓库的容量和产能限制。采用 **PULP_CBC_CMD 求解器**,通过分支割平面法求解,使得模型能够有效探索可行的分仓方案,最终实现了整体仓租成本的降低。最终发现求解时间设定为 400 秒时,此时为最优解,总成本为 6977267.16125076 元。

针对问题三,在多目标优化的背景下,本研究旨在利用**模拟退火算法**解决电商品类的多仓分配问题,以最大化仓储布局的整体收益。我们首先定义了一个综合收益函数,涵盖高级品类收益、件型一致性收益、品类关联度收益和总仓储成本,并为各收益项赋予不同权重,以合理平衡多目标收益,确保优化过程的全面考量。通过逐步降低温度的**模拟退火算法**,探索解空间以优化收益并控制成本,逐渐减少非优解的接受概率,引导解的收敛,最终保留当前最优解。总成本为 27,502,455.91 元。关联度收益为 390,167.564。高级品类收益达到了 7,860,000。件型一致性收益为 5,670,000,虽然仓租总成本上升但是各项业务指标兼顾考虑,能够更好的适应现实情况。

该方案在处理**非线性关系**和**动态变化**时表现出色,能够适应市场波动,提升预测的准确性。此外,模型的灵活性和系统化决策支持功能,使得资源配置更加合理,运营效率得到显著提升,能够在不同业务场景中推广,为企业提供科学依据和数据支持。

关键词: SARIMA 模型 随机森林回归模型 线性规划模型 模拟退火算法

目录

一、 问题重述	1
1.1 问题背景	1
1.2 问题提出	1
二、 问题分析	1
2.2 问题一的分析	2
2.3 问题二的分析	3
2.4 问题三的分析	3
三、 模型假设与符号说明	4
3.1 模型基本假设	4
3.2 符号说明	4
四、 问题一模型建立与求解	5
4.1 数据预处理	5
4.1.1 处理历史销量中的缺失值	5
4.2 库存量预测模型的建立与求解	6
4.2.1 数据与模型的适配性分析与处理	6
4.2.2 模型构建与求解逻辑	9
4.2.3 SARIMA 模型描述	9
4.2.4 基于网格搜索的自动参数优化	10
4.2.5 模型评估结果与分析	10
4.3 销量预测模型的建立与求解	11
4.3.1 数据与模型的适配性分析	11
4.3.2 模型构建与求解逻辑	11
4.3.3 随机森林回归模型描述	11
4.3.4 模型评估与参数检测	12
4.3.5 模型评估结果与分析	14
五、 问题二模型建立与求解	15
5.1 仓库分配模型的建立与求解	15
5.2 数据与模型的适配性分析	16
5.3 模型构建与求解逻辑	16
5.4 模型求解与结果分析	17
5.5 模型评估与应用	19
六、 问题三模型建立与求解	19
6.1 数据准备与处理	19
6.2 模型建立与求解过程	20

6.2.1 决策变量与目标函数设定	20
6.2.2 目标函数	20
6.2.3 收益计算逻辑	20
6.2.4 约束条件	22
6.2.5 决策变量约束	22
6.2.6 求解过程	22
6.2.7 最优解输出	23
6.3 模型评估与分析	24
6.4 分析不同方案中各业务指标的表现	24
七、 模型分析检验	26
7.1 灵敏度分析	26
7.2 误差分析	27
7.2.1 问题一的误差分析	27
7.2.2 问题二的误差分析	27
7.2.3 问题三的误差分析	27
7.3 整体误差分析	27
八、 模型评价与推广	27
8.1 模型的优点	27
8.2 模型的不足	28
8.3 模型的推广	28
参考文献	29
附录	30

一、 问题重述

1.1 问题背景



电商企业的商品存储系统由多个仓库组成，商品的存放主要依赖于其属性（如品类和件型）的分类和标记，以实现更高效的库存管理。在分仓规划过程中，必须预测两个关键指标：库存量和销量。为了有效应对仓容和产能的限制，同时考虑品类库存管理的复杂性和整体成本，准确的预测结果对于决策者至关重要。这些结果将帮助他们合理分配未来的仓储资源，减少不必要的空间浪费。因此，在确定各品类的预测货量之后，如何进行品类的分仓规划成为供应链管理者的课题。

在制定最优的分仓方案过程中，需要全面考虑仓群的复杂性以及各个仓库的仓容和产能限制。关键指标包括仓容利用率、产能利用率和总仓租成本等。当前的仓储网络由 140 个仓库和 350 种品类组成，并提供了各品类的历史库存量和销量、不同仓库的相关信息（包括仓租日成本、仓容上限和产能上限）、品类之间的关联度（未出现的组合设为 0）以及不同品类的详细信息（如品类编码和件型）。因此，构建一个高效的模型来解决商品存放问题显得至关重要。对此我们建立模型解决下列问题：

1.2 问题提出

基于前述的研究背景，本题目给出了 5 个具有相关数据的信息附件和 1 个上传答案的参考附件。本题所要解决的问题如下：

问题一：建立货量预测模型，对该仓储网络 350 个品类未来 3 个月（7-9 月）每个月的库存量及销量进行预测，其中库存量根据历史每月数据预测月均库存量即可，填写表 1 的预测结果并放在正文中，并将完整结果填写在 result 表格文件中的“月库存预测结果”的表单中；销量需给出未来每天的预测结果，填写表 2 的预测结果并放在正文中，并将完整结果填写在 result 表格文件中的“日销量预测结果”的表单中。

问题二：假设当前限定每个品类只能放在一个仓库中，即一品一仓，各品类之间请基于问题 1 的预测结果建立规划模型，综合考虑多个业务目标，求得品类的分仓方案，包括：应使用哪些仓库，使用的仓库需存放哪些品类的库存。填写表 3 的分仓结果并放在正文中，并将完整品类分仓结果填写在 result 表格中的“一品一仓分仓方案”表单中。

问题三：现在为每个品类按照件型及高级品类进行打标（如附件 5），并放开一品一仓假设，即允许一个品类存放于多个仓库，但同一品类存放的仓库数量不能超过 3 个，并希望同伴型、同高级品类尽量放在一个仓库中。假设同一品类在不同仓库之间分布的库存量比例及出库量比例相同，当前业务的首要目标是最大品类关联度，同时兼顾其他指标。请基于问题 1 的预测结果建立规划模型，求得新的品类分仓方案，并分析不同方案中各业务指标的表现。填写表 4 的分仓结果并放在正文中，并将完整品类分仓结果填写在 result 表格中的“一品多仓分仓方案”表单中。

二、 问题分析

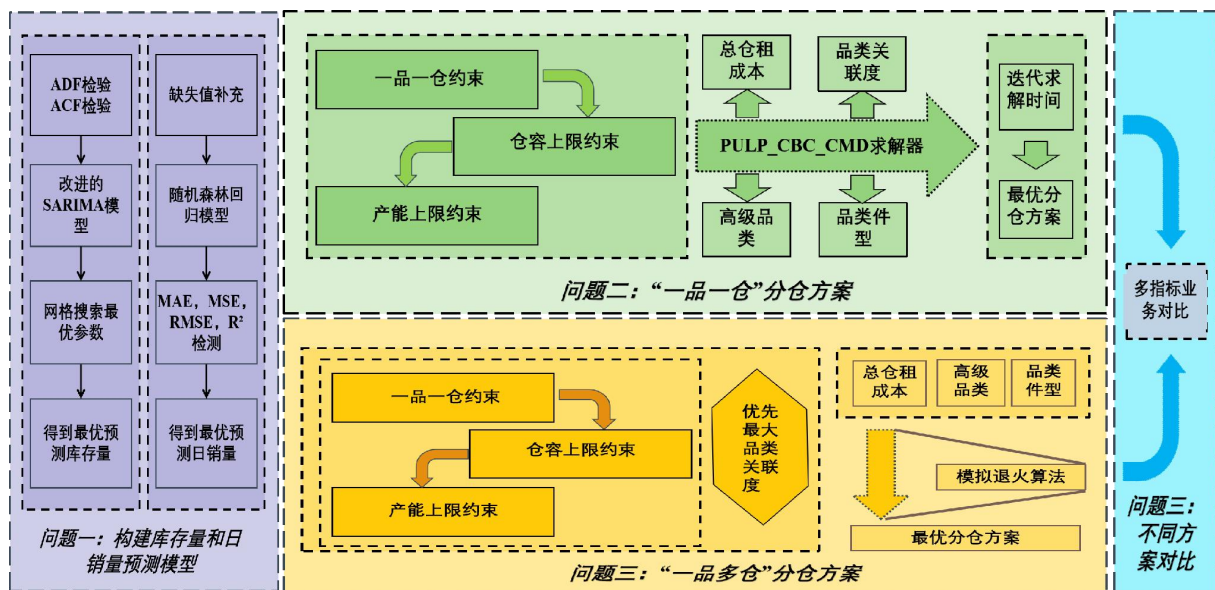


图 1 问题分析流程图

2.2 问题一的分析

在本次研究中，我们针对销量预测问题进行了深入的数据预处理和模型构建，以确保预测结果的准确性和可靠性。初步分析发现，历史销量数据中存在缺失值，这可能会影响后续模型的性能。因此，首先进行了数据的清洗和缺失值填补。

数据预处理：

在对历史销量数据的分析中，发现某些品类的销量记录不足 183 天，可能导致数据的完整性受到影响。为了提高模型的预测效果，我们采用**随机森林回归模型**对缺失值进行了填补。随机森林的优势在于其强大的非线性拟合能力和鲁棒性，能够有效应对时间序列数据中的波动性和异常值。通过提取日期特征（如日、月和星期），我们成功捕捉到了销量的时间规律，为后续的模型训练提供了高质量的输入数据。

经过多轮的交叉验证，随机森林模型在填补缺失值的过程中成功模拟了品类销量的历史趋势，形成了一个覆盖 183 天的完整数据集。这一过程不仅保留了原始数据的重要趋势，还揭示了更多的细节波动，显著提升了数据的分析价值。

库存量预测模型的建立：

在构建库存量预测模型时，我们对附件数据进行了详细的适配性分析。通过 ADF 检验和自相关性分析，发现多个品类的数据表现出明显的非平稳性和季节性特征。针对这些特征，我们采用线性插值法处理缺失数据，并通过差分操作将非平稳序列转化为平稳序列，为建立 SARIMA 模型打下了基础。

SARIMA 模型因其能够有效处理时间序列中的季节性波动，成为我们库存量预测的理想选择。通过网格搜索优化模型参数，我们确保了每个品类的 SARIMA 模型都能够准确捕捉库存量的趋势与季节性特征，为后续的库存管理提供了可靠的依据。

销量预测模型的建立：

在销量预测部分，我们使用**随机森林回归模型**对未来 3 个月的销量进行预测。通过对历史销量数据的深入分析，我们发现两段时间的数据存在明显的季节性和周期性波动，随机森林模型在处理这些复杂数据时展现出良好的鲁棒性和预测能力。

模型评估过程中，我们采用多种指标如 MAE、MSE、RMSE 和 R^2 ，全面验证了模型的预测性能。绝大部分品类的预测结果表明，模型能够有效捕捉销量变化趋势，且预测误差保持在合理范围内，进一步为后续的库存管理和分仓规划提供了坚实的支持。

通过本次问题一的分析与求解，我们不仅成功填补了历史销量数据的缺失值，还构建了适应性强的库存量和销量预测模型。整个过程强调了数据预处理的重要性以及模型选择的合理性，为未来的预测任务奠定了良好的基础。

2.3 问题二的分析

在电子商务的仓储管理中，合理的品类分仓决策是提升仓库管理效率和降低仓储成本的关键。参考《电商物流分仓备货优化研究》^[3]中的思路，问题二旨在解决“一品一仓”的品类分仓问题，即每个品类需唯一分配到某个仓库。该问题具有以下特点：

业务背景与问题性质：在分仓场景中，电子商务企业通常面对复杂的品类和仓库布局。合理分仓可以防止仓库资源浪费，并提升管理便捷性。但因仓库容量和产能的限制，以及仓租成本的存在，每个品类的仓库选择需考虑多个因素，包括仓库的存储能力、出库能力和运营成本，增加了决策难度。由于需要满足仓库的容量和产能约束，且目标是最小化仓租成本，本问题可建模为一个具有约束条件的资源分配优化问题。

核心优化目标：问题二的核心目标在于通过最小化总仓租成本，为每个品类分配一个仓库，同时确保在满足仓库仓容和产能约束的前提下实现仓库资源的合理利用。将总仓租成本降至最低有助于降低运营开支，并确保资源配置符合公司业务需求。

关键约束条件如下：

1. 一品一仓约束：确保每个品类仅分配到一个仓库，以简化仓储管理并优化仓库资源利用。

2. 仓容上限约束：每个仓库的品类库存总量不得超过仓库容量上限，以避免仓库超载、影响存储安全和管理效率。

3. 产能上限约束：仓库每日的出库量不得超过仓库的产能上限，以确保在订单量高峰期满足履约需求，防止出库瓶颈。

4. 模型的适用性与求解方法：本问题的解答过程采用了线性规划模型，适合此类资源分配问题。线性规划通过简单的约束定义和目标函数构建，能够快速求解大规模的资源分配问题，提供高效、接近最优的解。特别地，在求解器的选择上，**分支割平面法**被用于处理整数解要求，有助于在求解过程中快速收敛，找到最优分仓方案。此外，通过增加计算时间和迭代次数，可以更接近理论最优解，使得解的质量和求解效率得到良好平衡。

5. 业务决策的参考指标：为进一步评估分仓方案的合理性，模型解答过程还综合考虑了仓容利用率、产能利用率、总仓租成本、品类分仓数和品类关联度等指标。合理的仓容和产能利用率确保仓库资源有效分配；较低的仓租成本降低运营支出；适当的品类分仓数控制管理复杂度。这些指标共同构成了对模型性能和分仓效果的多维评价基准，有助于决策者在多目标之间进行权衡。

综上所述，问题二的品类分仓问题是一个典型的**线性规划**资源分配优化问题，具有约束条件多、成本优化复杂的特点。通过构建线性规划模型、利用分支割平面法求解、在较短时间内找到接近最优解，该模型能够满足仓储分配的业务需求，同时提供科学合理的分仓决策支持。

2.4 问题三的分析

在问题三中，我们需要解决的核心问题是**电商品类的多仓分配优化，以最大化仓储布局的整体收益**。针对电商行业多品类、多仓储的特点，传统单仓存储模式在满足复杂市场需求和快速响应客户需求方面存在一定的局限性。因此，如何在满足各项业务需求的基础上合理分配仓储资源，成为了一个具有挑战性的多目标优化问题。

为实现这一目标，我们提出了一个**综合收益函数**，涵盖了影响仓储布局的关键因素：高级品类收益、件型一致性收益、品类关联度收益和总仓储成本。高级品类收益旨在提升高价值商品的存储与管理效率；件型一致性收益则注重于提升相似件型商品的运营效率；品类关联度收益通过优化相关商品的集中存放以提高交叉销售的机会；而总仓储成本则直接影响仓储方案的经济性。通过赋予各收益项和成本项不同的权重，我们希望在整体收益最大化和成本控制之间实现合理的平衡。

本题目中，选择**模拟退火算法**来求解多仓分配优化问题，主要是因为其在复杂解空间中的强大寻优能力。模拟退火算法通过逐步降低温度的方式减少非优解的接受概率，从而引导解的收敛。该算法具有探索解空间和避免局部最优的特性，特别适合于多目标优化问题。在求解过程中，通过多次迭代和温度递减控制，模拟退火算法能够在庞大的解空间中逐步接近全局最优解，并通过持续的优化改善方案的整体收益。

为了确保模型的实际应用性，我们在模型构建中注重数据的完整性和合理性。具体而言，通过数据准备与处理步骤加载仓库信息、品类关联度、日销量预测、月库存量预测以及件型和高级品类信息，保证输入数据的准确性。同时，将仓储成本、仓容上限、产能上限等从日单位转为月单位，简化了计算维度，确保所有数据在相同的时间单位内进行比较和运算。通过数据结构设计，我们也提高了收益计算的效率和模型的可维护性。

综合来看，本问题分析强调了多仓分配在优化电商仓储资源方面的重要性。通过构建多目标综合收益函数并使用模拟退火算法求解，我们的模型能够提供灵活、平衡的仓储布局方案，以提高整体的运营效率和市场适应性。这一方案不仅适用于不同品类的电商品仓储规划，同时为电商企业在实际运营中的决策提供了有价值的参考。

三、 模型假设与符号说明

3.1 模型基本假设

(1) **数据真实性假设**：通过假设所提供的数据是准确和可靠的，没有严重的错误或缺失。

(2) **季节性规律假设**：假设库存量的季节性变化规律是可预测的，并且在未来会保持一致。

(3) **线性关系假设**：假设仓租成本、仓容和产能与分配方案之间存在线性关系。

(4) **静态资源假设**：假设仓库的容量和产能是固定的，并且在预测期间不会发生变化。

(5) **数据稳定性假设**：假设品类之间的关联度在预测期间保持稳定。每个品类的件型在预测期间保持不变。每个品类的高级品类分类在预测期间保持不变。

(6) **月份固定假设**：假设每个月固定为 30 天，从而方便仓容上限，产能上限，仓租日成本转换为以月为单位进行计算。

3.2 符号说明

表 2 符号说明

符号	含义
x_{ij}	品类 <i>i</i> 是否分配到仓库 <i>j</i> （1 表示分配，0 表示不分配）
R_j	仓库 <i>j</i> 的月租成本
I_i	品类 <i>i</i> 的库存量
S_i	品类 <i>i</i> 的销量

C_j 仓库 j 的月仓容上限 P_j 仓库 j 月产能上限

四、 问题一模型建立与求解

4.1 数据预处理

4.1.1 处理历史销量中的缺失值

在进行销量预测之前，我们需要对历史销量数据进行预处理，以确保数据的完整性和准确性。通过对所提供的历史销量数据进行分析发现，部分品类的销量记录存在缺失，即其记录天数少于 183 天。这种数据缺失可能会对预测模型的效果造成不利影响，因此我们决定采用合理的方法进行数据补全，为模型预测提供高质量的输入。图 1 为历史销量的天数统计。

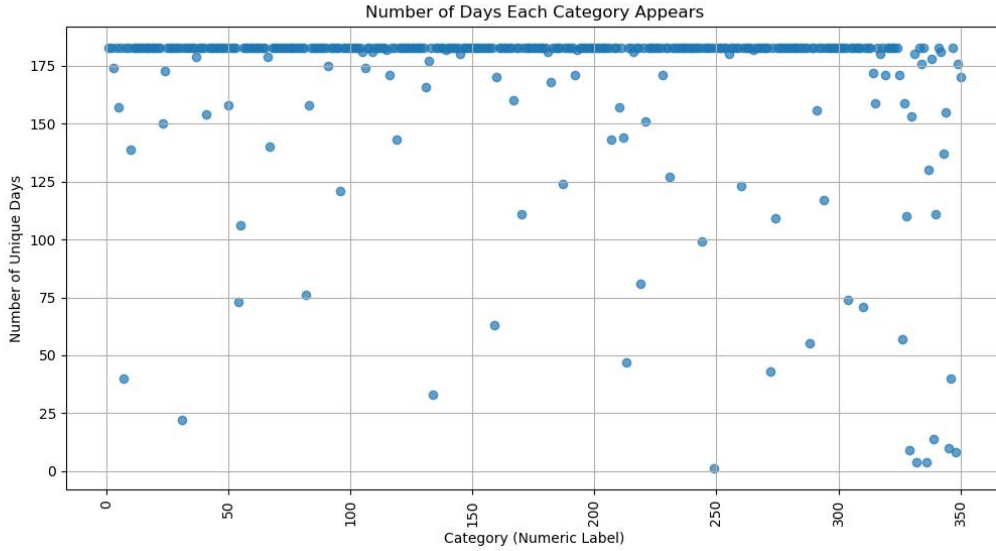


图 2 历史销量天数统计

针对这些缺失值的补全，我们选择了随机森林回归模型作为填补方法。随机森林回归是一种基于多棵决策树的集成学习算法，能够捕捉数据中的非线性关系，并在面对不规则缺失数据时表现出色。与传统的均值填补或线性插值方法相比，随机森林模型更适合处理时间序列数据中的波动性问题。此外，该模型的鲁棒性较强，能够有效应对噪声数据和异常值对填补结果的干扰。

随机森林的回归公式为：

$$\hat{y} = \frac{1}{N} \sum_{i=1}^N T_i(x) \quad 1)$$

其中， $\hat{y}$ 是最终的预测值， N 是随机森林中的决策树数量， $T_i(x)$ 是第 i 棵树对输入 x 的预测。

在数据处理的具体操作中，我们对所有品类的数据进行了清洗和重构，并与完整的日期范围进行对齐。对于缺失值的部分，我们提取了日期特征，如日、月和星期，以捕捉销量的时间规律。这些特征与已知的销量数据一起作为模型的输入，用于训练随机森林回归模型，并通过该模型预测缺失日期的销量。

在模型训练过程中，我们采用了交叉验证的方法评估模型的性能，确保其在不同数据集上的预测准确性。经过多次验证，模型成功模拟了品类销量的历史趋势，并将预测结果作为缺失数据的补充。通过这一过程，我们生成了覆盖 183 天的完整历史销量数据集。

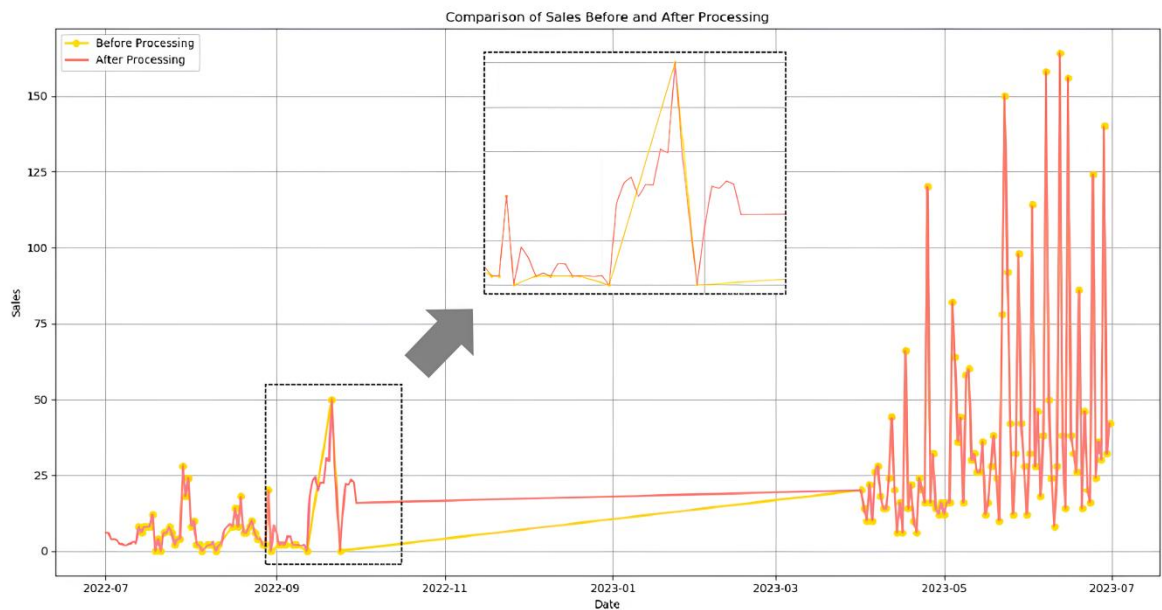


图 3 品类 category10 数据预处理前后对比图

图 2 是展示品类 category10 数据预处理前后对比，在 2022 年 9 月份的销售数据中，可以明显观察到，经过数据预处理后（淡红色曲线），出现了多个拐点和波动，而处理前（黄色曲线）相对平稳。这些拐点的增多说明预处理不仅保留了原始数据的重要趋势，还揭示了更多的细节波动，使得数据的分析价值进一步提升。从局部放大图的展示可以看到，数据处理前的销量曲线变化较为单一，而处理后的曲线在同一时间段内呈现出更为复杂的变化。这些变化使得销售数据的关键点得以更加清晰地展现。整体来看，数据预处理的过程增强了数据的使用性和可解释性。通过揭示更多的销售拐点和趋势变化，预处理后的数据能够更好地支持未来销售趋势的预测与异常检测，提高了数据的分析价值。这表明预处理不仅是数据清洗的步骤，更是发掘数据潜力的重要手段。

这一数据预处理过程为后续的预测和优化模型提供了可靠的数据支持。完整且准确的历史数据不仅提高了预测模型的精度，也为品类的合理分仓规划奠定了坚实的基础。

4.2 库存量预测模型的建立与求解

4.2.1 数据与模型的适配性分析与处理

在参考《电商销量预测及其分仓备货策略研究》^[2]这篇文献后对问题一有了较为清晰的思路，先分析附件 1 的数据可以发现，缺失了 2022 年 9 月，10 月，11 月的数据，本文选择先对所有数据进行特征分析，即对数据进行特征分析，观察各品类是否具有季节性或周期性变化，为后续建模提供更多信息。

（1）时间序列稳定性（平稳性）分析：

我们首先采用了 ADF 检验，该检验是一种用于检验时间序列数据是否为平稳序列的统计方法。时间序列的平稳性很重要，因为许多时间序列模型要求输入数据是平稳的。若数据不平稳，需要通过差分或其他方法将其转换为平稳序列。

ADF 检验的回归模型表示为：

$$\Delta X_t = \alpha + \beta t + \phi X_{t-1} + \sum_{i=1}^p \theta_i \Delta X_{t-i} + \epsilon_t \quad 2)$$

Δ 是差分运算符，即 $\Delta X_t = X_t - X_{t-1}$ ， α 是常数项， β 是趋势项，允许检验序列中是否存在趋势。 ϕ 是待检验的参数，若 $\phi=0$ ，则表示存在单位根，序列非平稳， θ_i 是滞后项的系数， ϵ_t 是白噪声误差项。

(2) 设置原假设与备择假设：

- 原假设 (**H0**)：设置序列存在单位根，即序列是非平稳的。
- 备择假设 (**H1**)：设置序列没有单位根，即序列是平稳的。

p-value 是在原假设 H0 为真时，观察到的统计量等于或更极端的概率。可以用以下公式表示：

$$p - value = P(T \leq t|H_0) \quad 3)$$

其中，T 是检验统计量的随机变量，t 是在样本中计算出的统计量。

判断标准：

p-value < 0.05：拒绝原假设 (H0)，认为序列是平稳的。

p-value ≥ 0.05：无法拒绝原假设 (H0)，认为序列是非平稳的。随机抽取的 60 种品类的检验结果如图 1 所示。其余结果见附件。

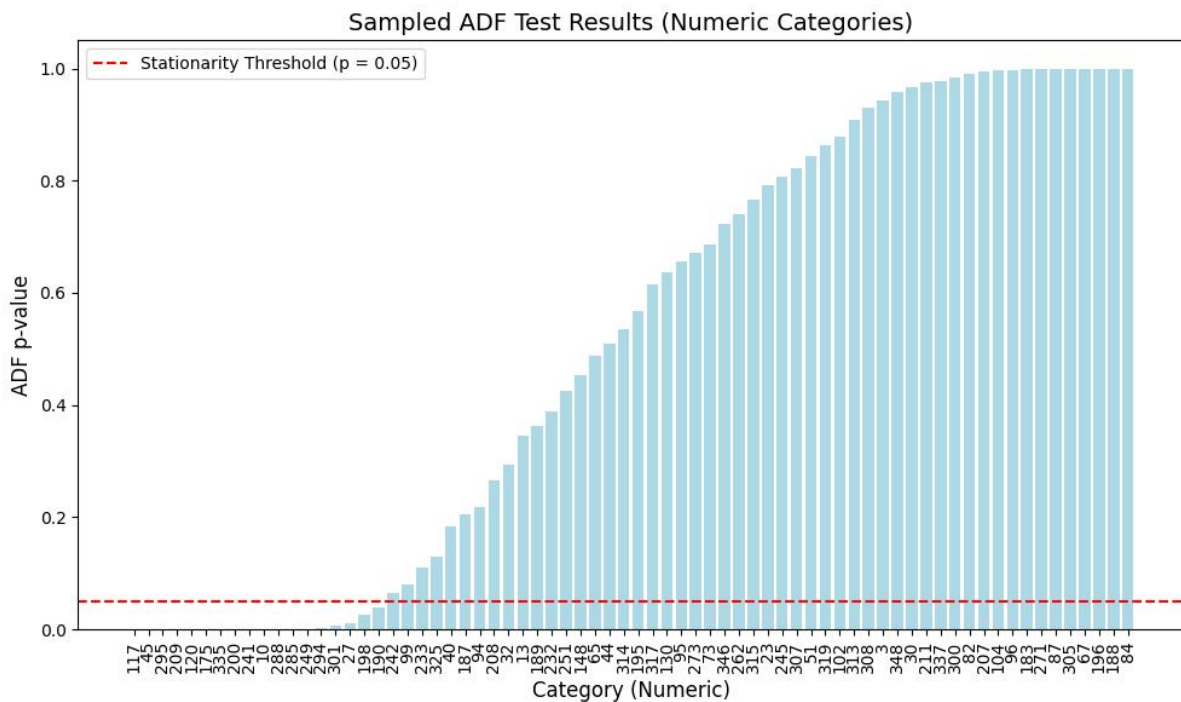


图 4 部分数据的 ADF 检验图

这张 ADF 检验图表明库存量的时间序列数据中存在明显的非平稳性，其中超过一半的品类表现出显著的趋势或波动（右侧的高 p-value 类别）。这些类别的 p-value 远高于 0.05，说明这些数据的时间序列非平稳。这些非平稳序列可能代表了存在趋势性或季节性波动的库存品类。

(3) 周期性与自相关性检测：

我们下一步采用了自回归条件异方差（Autoregressive Conditional Heteroskedasticity, ACF）分析，该分析是一种时间序列分析方法，主要用于检测序列中是否存在条件异方

差现象，也就是通过观察时间序列的自相关性来判断数据在不同时间段是否存在显著的波动差异。

本文对各品类进行 ACF 分析进而判断数据的自相关性，部分分析结果如图 2 所示。其余结果见附件。

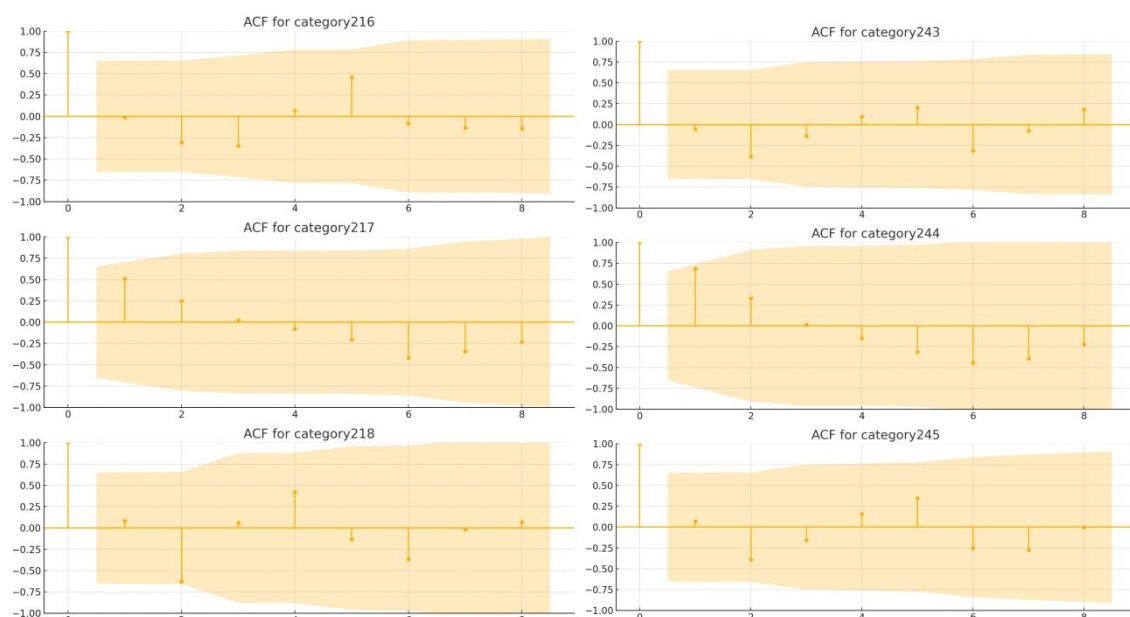


图 5 部分品类的 ACF 图

多个品类（如 216, 217, 219）显示出强烈的滞后 1 自相关，这意味着它们之间可能存在短期的强趋势或季节性依赖。例如 218, 245 这样的品类，滞后 2 和滞后 4 的自相关值显著，可能存在短周期的波动性。例如 243 品类别，似乎呈现出平稳的特征，自相关值较低且快速下降。

（4）缺失值处理：

在附件 1 的数据中，2022 年 9 月、10 月和 11 月的数据缺失。由于时间序列数据通常具有连续性，本文选择线性法插值可以有效地在缺失值与已知值之间建立一种线性关系，以尽可能保持数据的连续性和一致性。

通过对时间序列的特征分析，如平稳性检验和自相关性分析，我们发现多个品类表现出明显的非平稳性及季节性特征。线性法插值可以在这种趋势或季节性波动的基础上进行推断，使得填补的缺失数据能够反映出数据的整体趋势，而不是简单地插入平均值或中位数。

ADF 检验结果显示，许多品类的数据存在显著的非平稳性，p-value 远高于 0.05，表明这些序列可能具有趋势或季节性波动。线性插值在这种情况下，能够在已知数据点的上下文中平滑过渡，反映数据的潜在趋势。同样，ACF 分析结果表明，部分品类存在明显的自相关性。这意味着前期的数据对后期的值有一定的影响，而线性插值可以利用这些信息，确保插值的合理性和有效性。

与其他插值方法相比，线性法插值不需要假设数据的具体分布形式或其他复杂结构。它仅依赖于已有数据点的直线连接，这减少了对模型假设的依赖，使得插值结果更具普适性。

（5）差分处理：

在时间序列分析中，确保数据的平稳性是建立有效模型的前提条件。正如前文所述，经过 **ADF 检验** 的分析结果显示，多个品类的库存量数据存在显著的非平稳性，这使得

我们需要采取措施将这些非平稳序列转化为平稳序列。为此，差分处理是一种常用且有效的技术。

差分的基本公式为：

$$Y_t' = Y_t - Y_{t-1} \quad 4)$$

其中， Y_t' 是差分后的时间序列， Y_t 是当前时刻的值， Y_{t-1} 是前一时刻的值。通过差分，可以有效消除时间序列中的趋势性，使其趋于平稳。

部分差分结果如下图所示。

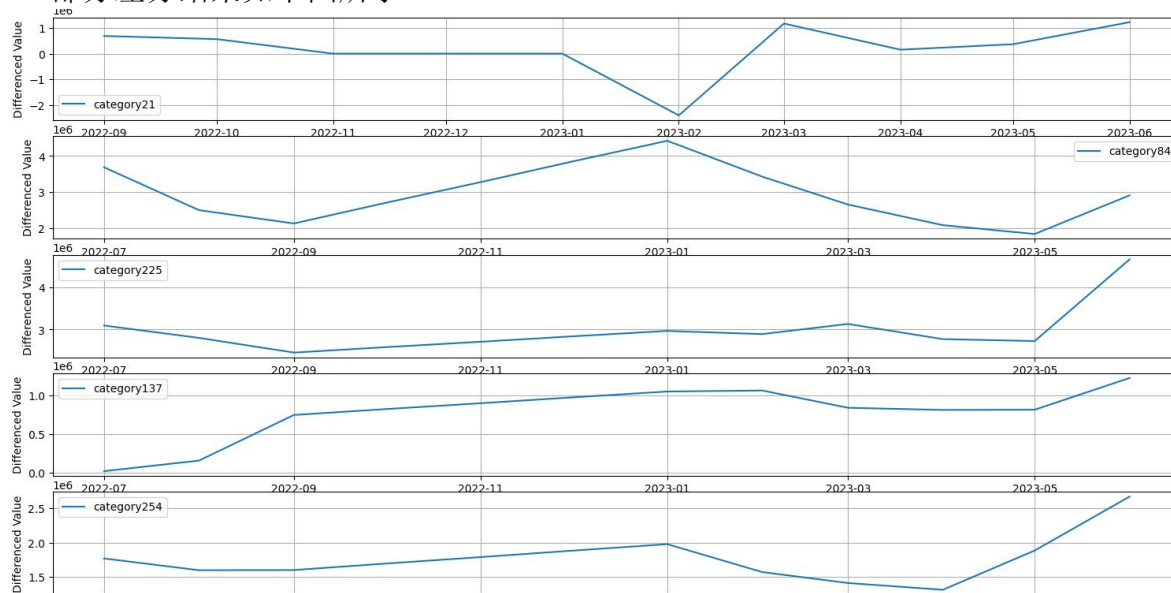


图 6 部分品类差分处理后的 ADF 检验

4.2.2 模型构建与求解逻辑

基于上述特征分析，SARIMA 模型是一个合理且有效的选择，主要原因包括：能够处理时间序列中的季节性波动（适合周期性明显的品类）。通过差分操作将非平稳序列转换为平稳序列，从而满足建模需求。能够捕捉短期自相关性，通过 AR 和 MA 机制来处理这些特征。季节性部分使得 SARIMA 模型在面对缺失数据或波动性强的数据时有更强的鲁棒性。对于分析中提到的这些库存品类的时间序列数据，SARIMA 模型能够综合处理非平稳性、周期性、趋势性以及自相关特征。因此，本文基于 SARIMA 季节性自回归积分滑动平均模型建立库存量预测模型。

4.2.3 SARIMA 模型描述

SARIMA 模型通常用记号表示为 $SARIMA(p, d, q)(P, D, Q)[m]$ ，其中： p, d, q 分别是 ARIMA 模型中自回归（AR）、移动平均（MA）阶数和差分次数。 P, D, Q 是季节性部分的自回归（SAR）、差分（SD）、和移动平均（SMA）阶数^[1]。 m 是季节的长度（周期），例如对于每年重复的月度数据， $m = 12$ 。

SARIMA 模型是季节性部分与非季节性部分的结合，定义为：

$$\Phi_P(B^m)\phi_p(B)(1-B)^d(1-B^m)^D y_t = \theta_Q(B^m)\theta_q(B)\epsilon_t \quad 5)$$

其中：

y_t 表示时间 t 的观测值。

ϵ_t 表示白噪声或随机误差项（通常假设为正态分布）。

B 是滞后算子（lag operator），即 $B y_t = y_{t-1}$ 。

$\phi_p(B)$ 和 $\Phi_p(B^m)$ 分别是非季节性和季节性自回归部分的多项式。
 $\theta_p(B)$ 和 $\Theta_Q(B^m)$ 分别是非季节性和季节性移动平均部分的多项式。
 $(1 - B)^d$ 是非季节性差分，用来使数据平稳。
 $(1 - B^m)^D$ 是季节性差分，用来消除季节性成分。

4.2.4 基于网格搜索的自动参数优化

在本研究中，为了提升库存预测的准确性，我们使用了 **SARIMA（季节性自回归积分滑动平均）模型**。SARIMA 模型的效果在很大程度上依赖于参数的合理设置，因此我们采用了**网格搜索（Grid Search）算法**，对模型的参数进行自动优化。

由于不同品类的库存数据具有各自的趋势和季节性特点，因此每个品类的 SARIMA 模型参数可能不同。为了确保模型在每个品类上的适用性，我们采用网格搜索（Grid Search）对参数进行自动优化。网格搜索遍历所有可能的参数组合，并通过比较每个组合对应的 AIC（Akaike 信息准则）值，选择使 AIC 最小的参数作为最优参数。

AIC 是衡量模型复杂度与拟合效果的标准，计算公式为：

$$AIC = 2k - 2\ln(L)$$

其中，k 为模型参数个数，L 为最大似然估计。AIC 值越小，模型的拟合效果越好，同时避免了模型过拟合。

网格搜索实现步骤：

- 1.定义参数搜索空间：p, d, q: 在[0,1,2]之间搜索。P, D, Q: 在[0,1]之间搜索。s: 设为 12（月度数据的时间周期）。
- 2.遍历参数组合：使用 sklearn 库中的 ParameterGrid 函数生成所有可能的参数组合，并逐一尝试拟合 SARIMA 模型。
- 3.选择最优参数：对每个参数组合，计算对应的 AIC 值。最终选取 AIC 最小的参数组合作为该品类的最优参数。
- 4.拟合最优模型并预测：使用最优参数拟合模型后，对未来 3 个月进行滚动预测，并将预测结果用于更新库存数据。

4.2.5 模型评估结果与分析

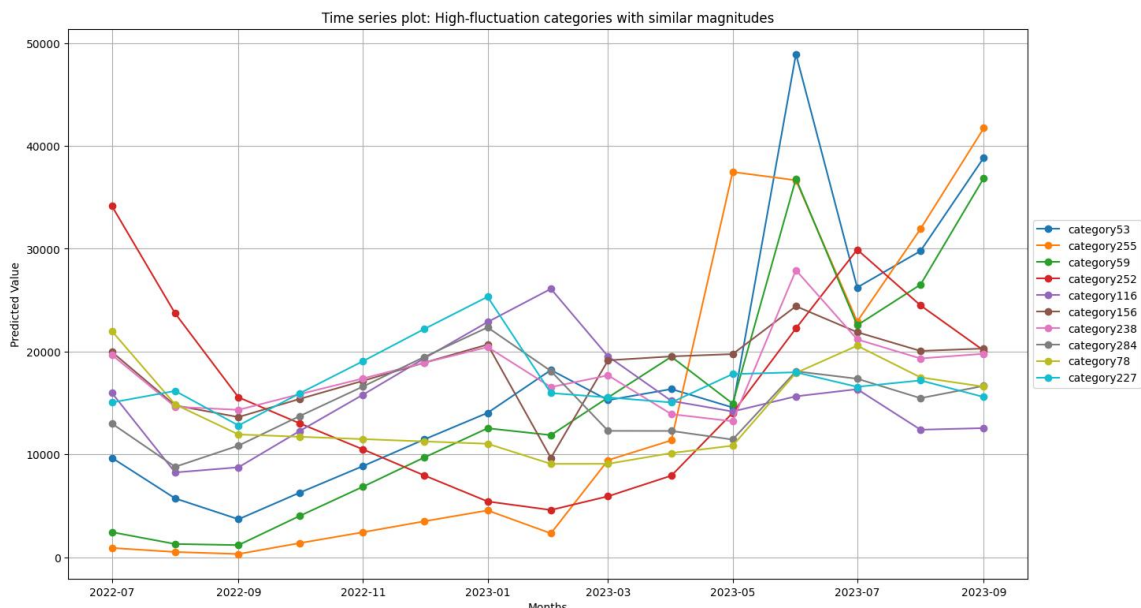


图 7 部分品类的时间序列预测图

基于参数优化的 SARIMA 模型在此预测中表现出色，成功捕捉到大部分商品类别的趋势与季节性特征。对于需求波动剧烈的类别（如 category53 和 category255），模型在捕捉其整体趋势上表现良好：在年中达到需求高峰，之后逐渐回落。这表明，模型成功识别了年中的季节性需求高峰。未来三个月的预测结果表明，不同类别的需求变化趋势各异，为库存管理提供了有效的参考。对于波动大的类别，建议灵活调整库存策略，平稳类别则可采用稳定库存策略以降低成本。

4.3 销量预测模型的建立与求解

在本次销量预测任务中，我们使用了**随机森林回归模型**对未来 3 个月（2023 年 7 月 1 日至 9 月 30 日）每天的销量进行预测。该模型基于**历史销量数据**和**日期相关特征**进行训练，以捕捉销量随时间变化的趋势和周期性波动特性。我们使用的数据涵盖了**2022 年 7 月 1 日至 9 月 30 日**和**2023 年 4 月 1 日至 6 月 30 日**两个时间段的销量记录。

4.3.1 数据与模型的适配性分析

分析历史销量数据后发现，这两个时间段的数据存在季节性和周期性波动。例如，7 月至 9 月可能是消费高峰期，而 4 月至 6 月可能为淡季，销量变化规律较为复杂。此外，由于数据来自两个不连续的时间段，存在一定的不规则性和缺失值。因此，我们选择了随机森林回归模型，它适用于处理非线性数据，并能够在应对数据缺失和噪声方面表现出良好的鲁棒性。

4.3.2 模型构建与求解逻辑

为了准确预测未来的销量，我们对各品类构建了日期特征和滞后特征，并使用随机森林回归模型进行训练和预测。

日期特征的构建：

日（day）：捕捉每月不同日期的销量规律。

月（month）：反映不同月份之间的季节性波动。

星期几（weekday）：识别工作日和周末的销量差异。

滞后特征的构建：

滞后 1 天、7 天、14 天和 30 天的销量值：这些特征帮助模型捕捉销量的短期和长期周期性趋势，如每周的销售高峰或每月的变化。

训练与测试集划分：

我们将数据按时间顺序划分为训练集和测试集，以避免打乱时间序列的结构。训练集用于模型的学习，测试集用于验证模型的预测能力。

逐步预测未来销量：

在训练完成后，使用最后 30 天的历史数据作为基准，通过逐步生成滞后特征，对未来 3 个月内每天的销量进行连续预测。

4.3.3 随机森林回归模型描述

设在某日 t 的销量为 y_t ，日期的日、月和星期几分别为 d_t, m_t, w_t 。滞后 1 天、7 天、14 天和 30 天的销量分别表示为 $\text{lag}_1, \text{lag}_7, \text{lag}_{14}, \text{lag}_{30}$ 。对于未来某日 $t+1$ ，模型的输入特征向量为：

$$[X_{t+1} = [d_{t+1}, m_{t+1}, w_{t+1}, \text{lag}_1, \text{lag}_7, \text{lag}_{14}, \text{lag}_{30}]] \quad 7)$$

基于这些输入特征，随机森林回归模型生成的预测值为：

$$\widehat{y_{t+1}} = f(X_{t+1}) \quad 8)$$

其中，f 是随机森林回归模型构建的回归函数。

4.3.4 模型评估与参数检测

为了确保模型的准确性和稳健性，我们在模型训练完成后，对每个品类在测试集上的预测结果进行了评估，采用了多种评估指标来验证模型的性能：

MAE（平均绝对误差）：

测量预测值与实际值之间的平均绝对误差。计算公式为：

$$MAE = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i| \quad 9)$$

MAE 越小，说明预测值与实际值的偏差越小。

MSE（均方误差）：

反映预测值与实际值之间误差的平方平均值，对大误差更加敏感。计算公式为：

$$[MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2] \quad 10)$$

MSE 较小时，表明模型对大幅偏差的预测较少。

RMSE（均方根误差）：

通过对 MSE 开平方得到 RMSE，直接衡量误差的大小。计算公式为：

$$[RMSE = \sqrt{MSE}] \quad 11)$$

RMSE 比 MSE 更容易直观地反映误差的大小。

R²（决定系数）：

评估模型对数据的拟合程度。其取值范围为 0 到 1，值越接近 1，表示模型对数据的解释能力越强。计算公式为：

$$R^2 = 1 - \frac{\sum_{i=1}^n (\hat{y}_i - y_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad 12)$$

其中， $\bar{y}$ 是实际值的平均值。

表 3 部分品类预测检验参数

品类	MAE	MSE	RMSE	R ²
Category304	1.3507911	3.748737	1.9361656	0.9009371
Category336	0.0136564	0.000541	0.0232735	0.9981120
Category339	0	0	0	1
Category345	0	0	0	1
Category31	0	0	0	1
Category348	0	0	0	1
Category332	0	0	0	1

对于 category304，模型表现良好， R^2 接近于 0.9，表明模型能够较好地拟合该品类的销量变化趋势，预测误差处于合理范围内。这表明该品类的数据具有较好的规律性，模型能够有效捕捉其销量的变化。

category336 的预测效果非常优异， R^2 达到 0.998，MAE 和 RMSE 几乎为 0，说明模型对该品类的销量变化进行了高度准确的拟合。模型的表现表明该品类的历史数据高度稳定，容易预测。

对于 category339、category345、category31、category348 和 category332，所有误差指标均为 0， R^2 为 1。这表明模型在这些品类上的拟合完全正确，预测值与实际值完全一致。这可能是由于这些品类的数据非常稳定，历史销量没有明显波动，或者测试集中销量保持不变，因此模型能够完美预测。

总体来看，模型在这些品类上的表现非常优异，尤其是对于稳定性较强的品类，预测结果完美。这表明随机森林回归模型在处理数据波动较小、具有较强规律性的品类时表现非常出色。对于 category304，虽然预测误差比其他品类稍大，但依然处于较高的预测水平，模型能够为后续的库存管理和分仓规划提供可靠的数据支持。

4.3.5 模型评估结果与分析

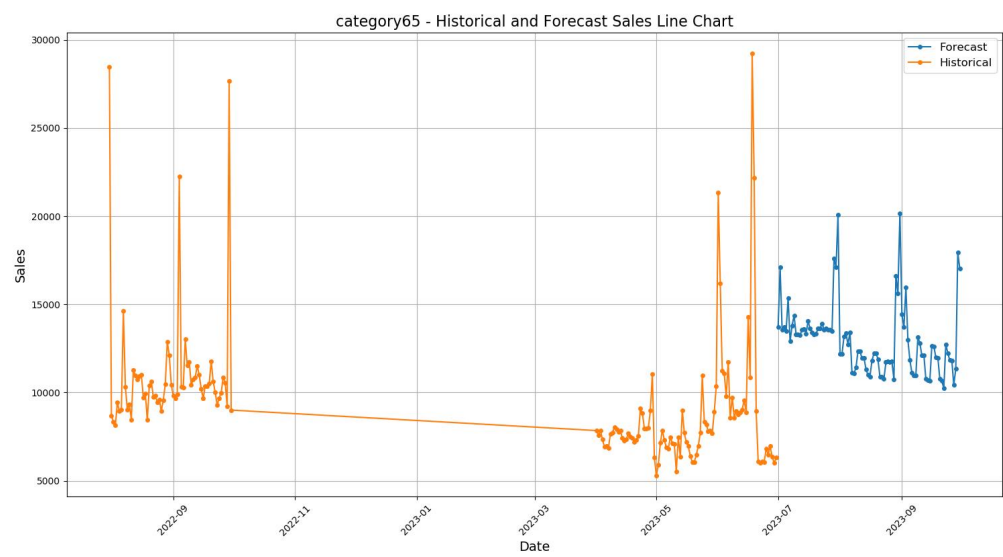


图 8 品类 category65 销量预测情况图

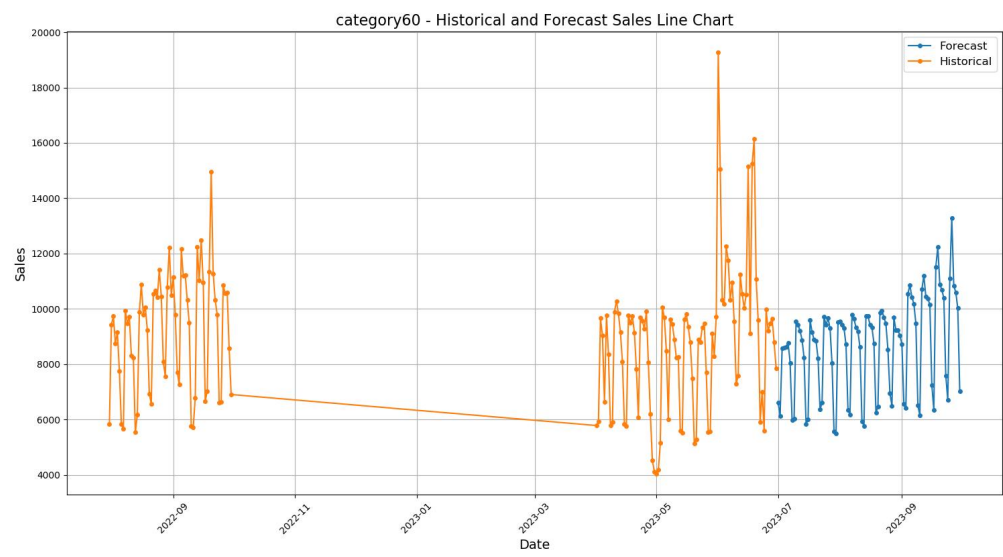


图 9 品类 category60 销量预测情况图

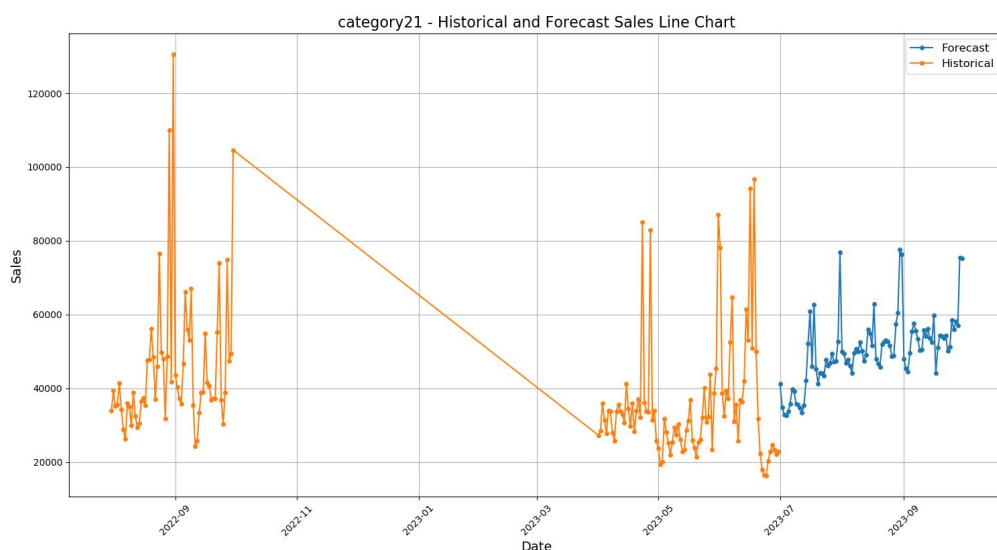


图 10 品类 category21 销量预测情况图

从三张预测结果图可以看出，模型在不同品类的销量预测中表现良好。对于 category65 和 category60，尽管历史销量中存在多次剧烈波动和极端峰值，模型的预测结果能够成功捕捉整体的趋势，并在预测区间内保持合理的波动。这表明模型在处理复杂的销量数据时具备一定的稳健性，能够有效避免因少数异常值导致的预测偏差。在 category21 中，模型不仅延续了销量的上升趋势，还在一定程度上反映了历史数据中的波动性，说明模型对销量变化的捕捉较为准确。尽管个别极端值未完全反映在预测结果中，但这种平滑处理使得预测结果更加稳健，为业务决策提供了可靠的数据支持。总体而言，模型在面对复杂数据时表现出色，既能把握长期趋势，又能控制波动范围，充分展现了其预测能力的优越性。通过进一步优化特征输入，模型在未来的实际应用中有望继续提升其准确性和适应性。

经过测试，我们发现随机森林回归模型在不同品类上的预测效果较为稳定。大部分品类在测试集上的 MAE 和 RMSE 保持在合理范围内，表明模型的预测误差较小。同时， R^2 决定系数大部分接近于 1，表明模型能够较好地拟合历史销量数据并捕捉其变化趋势。

多种评估指标的使用确保了模型的稳健性和准确性。在面对不连续的时间段和复杂的销量波动时，模型依然能够提供稳定的预测结果。这为未来 3 个月的库存管理和分仓规划提供了可靠的数据支持。

通过构建基于随机森林回归的销量预测模型，我们成功捕捉了销量的时间规律和滞后关系，并对模型进行了全面的评估。随机森林模型的非线性拟合能力、鲁棒性和逐步预测机制，使其在应对复杂时间序列和不规则数据时表现出色。多维度的评估指标验证了模型的准确性和稳健性，为后续的分仓规划和库存管理提供了强有力的支持。

五、 问题二模型建立与求解

5.1 仓库分配模型的建立与求解

在本次仓库分配任务中，我们构建了一个线性规划模型，以优化每个品类的仓库分配方案。该模型旨在通过最小化仓租成本，同时满足仓库的容量和产能限制，为每个品类分配一个仓库，从而提高仓库的管理效率。

5.2 数据与模型的适配性分析

通过分析现有的仓库和品类数据，我们发现仓库的仓容和产能限制是进行合理分配的关键因素。每个仓库的租金、容量和产能都影响着分仓决策。为了确保模型能够有效反映实际情况，我们采用线性规划方法，适合于解决这类资源分配问题。线性规划模型具备高效求解的特性，能够快速处理大规模的决策问题。

线性规划模型的适用性还体现在其对约束条件的灵活性。通过定义一品一仓、仓容上限和产能上限等约束，模型能够在确保可行解的前提下，实现目标函数的优化。这种特性使得模型在电商环境中具有较强的适应性

5.3 模型构建与求解逻辑

为了准确实现仓库分配，我们建立了一个包含目标函数和约束条件的线性规划模型。

目标函数：我们的目标是最小化总的仓租成本，目标函数形式为：

$$\text{Minimize } Z = \sum_{i \in I} \sum_{j \in J} R_j \cdot x_{ij} \quad (13)$$

其中， R_j 表示仓库 j 的月租成本， x_{ij} 是决策变量，表示品类 c 是否分配到仓库 j （1表示分配，0表示不分配）。

约束条件：模型中包含以下关键约束：

一品一仓约束：确保每个品类只能分配到一个仓库，避免资源的过度分散，简化管理流程。

$$\sum_{i \in I} x_{ij} = 1, \forall i \in I \quad (14)$$

仓容上限约束：限制每个仓库的库存量不超过其仓容上限，确保仓库的安全性和高效性。

$$\sum_{i \in I} x_{ij} \cdot I_i \leq C_j, \forall j \in J \quad (15)$$

产能上限约束：限制每个仓库的出库量不超过其产能上限，以保障在高峰期能够满足客户需求。

$$\sum_{i \in I} x_{ij} \cdot S_i \leq P_j, \forall j \in J \quad (16)$$

其中， I_i 为品类 i 的库存量， S_i 为品类 i 的销量， C_j 和 P_j 分别为仓库 j 的月仓容和月产能上限。

5.4 模型求解与结果分析

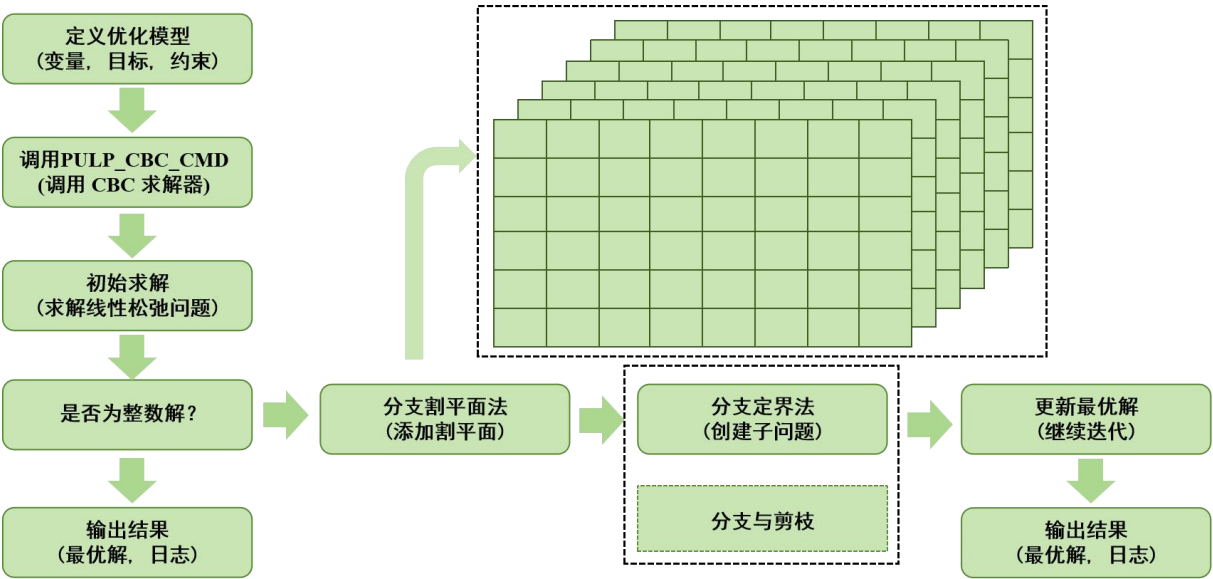


图 11 PULP_CBC_CMD 求解器原理图

在分仓模型中，使用分支割平面法^[4]能够有效地优化品类的仓库分配。该方法通过添加割平面来限制非整数解，同时保持所有整数解的可行性。例如，在进行初步求解后，如果发现某个品类的仓库分配并未能有效利用仓库的容量或者导致了产能超限，割平面将帮助排除不合适的分配方案。

分支过程的实施：

在模型实施过程中，求解器首先会对线性松弛问题进行求解，得到初步的最优解。接下来，根据当前的解进行分支，创建子问题以进一步探索可能的分仓方案。每次分支后，求解器会计算新的下界，通过比较当前最优解与新解的下界，判断是否需要继续深入搜索。通过这种方式，可以有效地缩小解空间，从而提高求解效率。

综合考虑的业务目标：

在进行品类分仓规划时，需综合考虑多个业务目标，包括仓容利用率、产能利用率、总仓租成本、品类分仓数及品类关联度等。这些指标共同决定了分仓方案的合理性。例如，合理的仓容利用率意味着仓库空间得到了充分利用，而产能利用率的提高则可以确保出库效率最大化。同时，降低总仓租成本和控制品类分仓数可以减少管理复杂度，提升整体运营效率。

迭代与优化结果：

在每次迭代中，求解器会根据新的割平面更新最优解，并在限制条件的框架下不断优化分仓方案。最终，求解器将输出最佳的分仓结果，包括使用的仓库和存放的品类，确保每个仓库的容量和产能限制得以满足。该结果将反映在结果文件中，并用于进一步的业务分析和决策支持。

在模型构建完成后，我们使用线性规划求解器对其进行求解。求解的过程确保了在满足所有约束条件的前提下，能够找到一个全局最优解。通过对结果的分析，我们可以明确每个品类应分配至哪个仓库，并了解各仓库的使用情况。

表 4 300 秒求解时间的规划结果参数表

指标	参数
Objective value	6977267.16125076
Lower bound	6059643.349

Gap	0.15
Enumerated nodes	14106
Total iterations	218943
Time (CPU seconds)	300.08
Time (Wallclock seconds)	300.08

这组结果展示了优化模型求解的性能与输出情况，目标值（Objective value）为 6,977,267.16，表示当前解对应的总成本，这是优化模型所要最小化的关键指标。也就是说，该解代表了一个可行的分仓方案，其总成本达到了接近最优的水平。下界（Lower bound）为 6,059,643.35，这是算法推断出的理论上可能达到的最低总成本，意味着在理想情况下，总成本不会低于该值。通过这些值的对比，我们可以评估当前解的质量。差距（Gap）为 0.15%，反映了当前解与理论最优解之间的相对差异。这么小的差距表明，当前解已十分接近最优解，进一步优化的潜力有限，表明算法已经在较高程度上实现了成本的优化。

求解过程中，共枚举了 14,106 个节点（Enumerated nodes），这说明算法在解空间进行了大量的探索，以找到最优解。这种枚举操作通常是分支定界（Branch-and-Bound）或类似算法的一部分，用于逐步缩小解的范围。与此同时，算法共迭代了 218,943 次（Total iterations），这反映了优化过程中的计算强度和复杂性——每次迭代都涉及更新解或寻找更优的解。如此多的迭代次数表明问题的规模较大，且求解器在逐步收敛的过程中付出了大量计算努力。

此外，整个求解过程的 CPU 时间和实际时间（Wallclock seconds）均为 300.08 秒，说明求解过程持续了大约 5 分钟，并且计算过程与实际耗时一致。这表明算法在求解过程中没有遇到显著的等待时间或资源竞争问题，从而保证了计算的高效性。这些指标综合表明，优化模型在有限的时间内成功地找到了一个高质量的解，并且该解与理论最优解之间的差异非常小。总体而言，这一分仓方案在成本控制上表现良好，表明算法在成本优化和求解效率之间取得了良好的平衡。

模型求解后的分仓方案能够有效降低整体仓租成本，同时合理利用仓库资源。我们对各品类的分仓结果进行了详细评估，并计算了总的仓容利用率和产能利用率，以进一步分析仓库分配的合理性。

表 5 三次不同求解时间的规划参数对比表

求解时间	目标成本	迭代次数	差距
300.08	6977267.16125076	218943	0.15
400.25	6823616.24231076	264415	0.13
500.17	6823616.24231076	379526	0.13

从表格数据可以看出，不同的求解时间对目标成本、迭代次数和差距（Gap）都有明显的影响。首先，目标成本随着求解时间的增加呈现下降趋势。在求解时间为 300.08 秒时，目标成本为 6977267.16125076；当时间延长至 400.25 秒时，目标成本下降至 6823616.24231076，表明求解器在更长的时间内找到了更优的解。然而，进一步增加求解时间到 500.17 秒后，目标成本保持在 6823616.24231076 不变，这说明求解器在延长的时间内未能找到更低的目标成本解，可能已经接近最优解。

此外，求解时间的增加伴随着迭代次数的显著增长。在 300.08 秒内，求解器共进行了 218,943 次迭代；延长到 400.25 秒时，迭代次数增加到 264,415 次；而在 500.17 秒时，

迭代次数进一步增加到 379,526 次。尽管迭代次数显著增多，但目标成本在 400.25 秒之后不再下降，这表明随着计算时间和迭代次数的增加，模型的优化效果趋于平缓，即解的质量提升空间有限。

差距（Gap）方面也展现出类似的规律。最初在 300.08 秒时，差距为 0.15%，表明当前解与理论最优解之间有 0.15% 的差异；当求解时间增加到 400.25 秒和 500.17 秒时，差距缩小至 0.13%，进一步接近理论最优解。这种差距的减少说明求解时间增加确实使解更接近最优解，但在 400 秒左右达到稳定，延长求解时间对差距的改善有限。

综上所述，这些数据表明，延长求解时间到 400 秒左右时，模型的求解效果已经达到较高的精度，目标成本和差距均趋于稳定，进一步增加求解时间并未带来明显的优化效果。这意味着 400 秒的求解时间在成本与精度上达到了较为理想的平衡，适合在实际应用中作为求解时间的参考值。

5.5 模型评估与应用

为确保模型的准确性与有效性，我们对分仓结果进行了综合评估。通过比较不同品类的分仓效率，我们能够识别出仓库资源利用的潜在问题，并对模型进行进一步的优化调整。此外，通过与实际运营数据的对比，验证模型的预测能力和适应性。

本模型的优势在于其结构简单、求解效率高且易于解释，适合在快速变化的电商环境中进行实时决策。未来，我们将继续探索更复杂的模型，以处理品类之间的关联性和变化性，从而提高分仓方案的优化程度。

六、 问题三模型建立与求解

在第三问中，我们的目标是在多目标优化的背景下，通过模拟退火算法求解电商品类的多仓分配问题，以最大化仓储布局的整体收益。为实现这一目标，我们首先定义了一个综合收益函数，涵盖高级品类收益、件型一致性收益、品类关联度收益及总仓储成本等关键因素，并为这些收益项和成本项赋予不同权重，以合理平衡各收益目标。

基于模拟退火算法的原理，我们逐步降低温度、探索解空间，从而优化收益并控制成本。这一过程通过减少非优解的接受概率，引导解的收敛，并在多次迭代中保留当前最优解，持续改善方案的整体收益。模拟退火算法在复杂解空间中寻优的能力，尤其适合多目标优化问题，确保有效平衡各收益目标。最终，模型输出的最佳分仓方案实现了多品类、多仓库的高效资源配置，为实际电商分仓布局提供了可操作和优化的参考方案。

6.1 数据准备与处理

在解决电商品类的多仓分配问题时，数据准备与处理是至关重要的第一步。我们的目标是最大化仓储布局的整体收益，因此必须确保输入数据的准确性和完整性，以便为模型提供高质量的基础。

首先，我们加载多个数据源，包括仓库信息、品类的关联度、日销量预测、月库存量预测以及品类的件型和高级信息。这些数据涵盖了仓储布局的核心要素，有助于全面分析和理解当前的资源配置。

为了便于后续计算，我们将仓租成本、仓容上限和产能上限从日单位转换为月单位。这一转换不仅能够简化后续的收益和成本计算，还能使得所有计算在同一时间维度上进行，提高模型的可比性和一致性。

在分析销量数据时，我们计算每个品类的最高月销量，确保在分仓规划中考虑到潜在的需求峰值。此外，通过获取每个品类的预测库存量最大值，可以为实际分仓决策提供更为合理的依据。

对关联度数据的处理则通过创建字典来实现，便于在计算收益时快速检索相关信息。这种数据结构的设计不仅提高了计算效率，还减少了代码的复杂度，使得整个算法更具可读性和可维护性。

综上所述，精心的数据准备和处理过程为后续的决策制定打下了坚实的基础，确保模型在多目标优化的求解中具备高效性和可靠性。

6.2 模型建立与求解过程

6.2.1 决策变量与目标函数设定

在本模型中，决策变量被定义为每个品类是否被分配到特定的仓库。这样的设计是为了灵活地应对每个品类在不同仓库中的需求特性，同时允许多个品类共享同一仓库。这一策略不仅能够提高仓储资源的利用效率，还能使得仓储布局更加灵活，适应市场需求的变化。

目标函数的构建是模型设计的关键部分。我们通过最大化综合收益函数来实现仓储布局的优化，该函数包括高级品类收益、件型一致性收益、品类关联度收益及总仓储成本。这一综合收益函数的设计有几个优点：

多目标平衡：通过为各项收益赋予不同的权重，我们能够在多个优化目标之间进行合理的平衡。例如，高级品类收益和件型一致性收益的权重设置使得品类在仓库中的分布既考虑到集中存储的优势，又保持了相同件型的管理便利性。

收益最大化：目标函数的设计直接与电商的盈利模式相关，确保我们的求解方向与实际业务目标一致。通过优化这些收益项，我们能够帮助企业实现更高的市场竞争力。

成本控制：在综合收益函数中，加入总仓储成本的考虑，使得模型不仅关注收益的增加，也重视成本的控制。这一设计能够有效避免因过度追求收益而导致的成本失控问题。

定义决策变量 x_{ij} 表示品类 i 是否分配到仓库 j ：

$$x_{cw} = \begin{cases} 1, & \text{品类}i\text{分配到仓库}j \\ 0, & \text{品类}i\text{不分配到仓库}j \end{cases} \quad (17)$$

6.2.2 目标函数

我们希望通过最大化以下综合收益函数 F ，使仓储分配达到最优效果，主要考虑品类关联度、件型一致性、高级品类收益和仓储成本的影响。目标函数可表示为：

$$\text{Maximize } F = w_1 \cdot H + w_2 \cdot U + w_3 \cdot A - w_4 \cdot C \quad (18)$$

其中：

H 表示高级品类收益；

U 表示件型一致性收益；

A 表示品类关联度收益；

C 表示总仓储成本；

w_1, w_2, w_3, w_4 为各项的权重系数。

6.2.3 收益计算逻辑

收益计算逻辑的设计旨在全面反映各类收益对整体仓储布局的贡献，确保模型在评估分仓方案时具备准确性和可靠性。

在计算高级品类收益时，我们关注品类集中存储在同一仓库的情况。这种设计有助于避免高级品类的分散，提高资源的使用效率。通过增加重复高级品类的收益，我们鼓励在仓储布局中实现高价值品类的集中存放，从而增强整体盈利能力。

高级品类收益计算：

高级品类收益 H 定义为将相同高级品类集中存储在同一仓库中的收益，以避免高级品类的分散：

$$H = \sum_j \sum_{i \in I_j} h_i \cdot x_{ij} \quad (19)$$

其中：

h_i 为品类 i 的高级品类收益值；

I_j 表示分配到仓库 j 的品类集合。

件型一致性收益则衡量相同件型品类的集中存储效果，促使管理过程更加便捷，提高物流效率。通过奖励相同件型的集中存放，我们不仅提高了库存管理的便利性，也降低了跨仓调度的需求。

件型一致性收益 U 衡量将相同件型的品类集中存储在同一仓库的效果，提高一致性：

$$U = \sum_j \sum_{i \in T_j} t_i \cdot x_{ij} \quad (20)$$

其中：

t_i 为品类 i 的件型一致性收益；

T_j 为分配到仓库 j 的同件型品类集合。

品类关联度收益计算的设计，通过考虑品类之间的关联性，帮助减少物流成本。关联度较高的品类放置在同一仓库中，可以降低配送成本，提高客户服务质量。这一设计思路的好处在于，它强化了仓储布局的灵活性，使得业务能够更好地应对市场变化。

品类关联度收益：

品类关联度收益 A 衡量关联度较高的品类放置在同一仓库中的情况，以降低履约成本：

$$A = \sum_j \sum_{i, k \in A_j} r_{ik} \cdot x_{ij} \cdot x_{kj} \quad (21)$$

其中：

r_{ik} 表示品类 i 和品类 k 的关联度；

A_j 为分配到仓库 j 的品类集合对。

总成本的计算则通过惩罚逻辑确保每个仓库的资源分配均衡，避免因某一仓库资源过剩或不足而导致的效率低下。这一设计能够有效提升仓储布局的稳定性，增强模型的实用性。

总仓储成本：

仓储成本 C 表示使用各个仓库带来的日成本：

$$C = \sum_j c_j \cdot y_j \quad (22)$$

其中：

c_j 为仓库 j 的日租成本；

y_j 为仓库 j 是否使用的指示变量，即 $y_j = 1$ 表示仓库 j 被使用，否则 $y_j = 0$

通过上述收益计算逻辑的合理设计，我们能够全面评估不同分仓方案的有效性，从而为最终的决策提供有力的数据支持。

件型一致性收益：

6.2.4 约束条件

(1) 仓容上限约束

仓容上限约束确保每个仓库的总库存不超过其存储容量 C_j ：

$$\sum_i p_i \cdot x_{ij} \leq C_j \cdot y_j \quad \forall j \quad (23)$$

其中：

p_i 表示品类 i 的预测库存量。

(2) 产能上限约束

产能上限约束确保每个仓库的出库量不超过其出库产能 P_j ：

$$\sum_i s_i \cdot x_{ij} \leq P_j \cdot y_j \quad \forall j \quad (24)$$

其中：

s_i 表示品类 i 的日销量预测值。

(3) 品类分仓限制

每个品类最多可存储在 3 个仓库内：

$$\sum_j x_{ij} \leq 3 \quad \forall i \quad (25)$$

6.2.5 决策变量约束

定义各决策变量的取值范围，确保每个变量的有效性：

$$x_{ij} \in \{0, 1\}, y_j \in \{0, 1\} \quad \forall i, j \quad (26)$$

6.2.6 求解过程

首先，为每一个品类 i 初始化一个随机的仓库分配方案。根据题目要求，每个品类最多可以分配到三个仓库，这样能够在满足库存和产能约束的情况下，为仓储布局提供一定的灵活性。具体的初始化步骤为：

对每个品类 i ，随机选择 1 到 3 个仓库进行初始分配。

生成一个“当前解”，该解包含了所有品类的仓库分配情况。

在模拟退火算法的迭代求解过程中，我们使用如下参数与方法进行优化：

温度参数：初始温度 T_0 设置为 1000，随着每次迭代降低温度，逐渐减少对不优解的接受概率，使解不断收敛^[5]。

迭代更新。

生成新解：在当前解的基础上，随机选择一个品类并对其分配仓库进行小幅调整，如增加或减少一个仓库，确保调整后仓库总数不超过三个。

计算新解的综合收益：利用综合收益函数计算新的分仓方案的收益。

接受准则：若新解的收益更高，则接受该解；若收益更低，则根据以下公式以一定概率接受：

$$P(\text{accept new solution}) = \exp\left(\frac{\text{current_profit} - \text{new_profit}}{T}\right) \quad (27)$$

其中：

current_profit 是当前解的综合收益。

new_profit 是新解的综合收益。

T 是当前温度（控制接受较差解的概率）。

这样可以在初始阶段探索更多解空间，避免局部最优。

温度递减：每次迭代后按冷却速率 $\alpha=0.99$ 更新温度 T，直到温度降到接近零或达到最大迭代次数。

算法如下：

Algorithm

1. $T_n = T_0$
 2. $V_n = \text{calculate_value}(I)[0]$
 3. For _ in range(M)
 4. $I[id] = 1 - I[id]$
 5. $V_{new} = \text{calculate_value}(I)[0]$
 6. $\Delta = V_{new} - V_n$
 7. if $\Delta < 0$ or $\text{random}() < e^{-\frac{\Delta}{T_n}}$:
 8. $V_n = V_{new}$
 9. else: $I[id] = I[id]$
 10. $T_n *= Co_R$
-

其中， T_n 表示当前温度， T_0 表示初始温度， V_n 表示当前个体适应度，I 表示待优化的个体，M 表示最大迭代次数，id 表示每一个品类， V_{new} 表示变异后个体的适应度值， Δ 表示新个体与当前个体的适应度差异， Co_R 表示降温过程。

6.2.7 最优解输出

在所有迭代完成后，最终输出最佳分仓方案和对应的综合收益值。此时的解是基于多目标收益优化和仓储约束下的最优方案，可以提高整体的分仓效益。

表 6 一品多仓的分仓结果

分仓方案	总成本	关联度收益	高级品类收益	件型一致性收益
一品多仓	27502455.91	390167.564	7860000	5670000

在本次模型求解中，得到了多个关键指标的结果，包括总成本、关联度收益、高级品类收益和件型一致性收益。以下是对每个指标的详细分析。

首先，总成本为 27,502,455.91 元。这一数值反映了电商在仓储管理中需要承担的费用，包括仓租、运营成本等。合理的总成本是保证企业盈利的基础，过高的成本会直接影响整体利润。因此，本模型在设定分仓方案时，致力于在优化收益的同时，控制总成本，以实现更为合理的资源配置。

接下来，关联度收益为 390,167.564。该收益指标体现了关联品类在同一仓库中的存放效果，能够减少配送成本，提高物流效率。通过优化品类的存放布局，企业可以降低跨仓调度的需求，从而提升整体运营效率。尽管这一数值相较于其他收益项较低，但它在整体仓储管理中的重要性不容忽视，尤其是在提高客户满意度和优化服务响应时间方面。

第三，高级品类收益达到了 7,860,000。这一指标突出了高价值品类集中存放所带来的经济效益，集中管理高级品类可以提高资源利用率和管理效率。通过优先配置这些品类到合适的仓库，企业能够在满足市场需求的同时，最大限度地发挥其经济效益，进而提升整体盈利能力。

最后，件型一致性收益为 5,670,000。这一收益指标反映了将相同件型的商品集中存放的效果，能够提高管理的便捷性，降低库存管理成本。件型一致性使得仓库运营更加高效，有助于实现快速周转和准确的库存管理，从而提升整体供应链的灵活性。

综上所述，本次结果分析显示，模型不仅成功优化了总成本，同时也通过多项收益的合理配置实现了整体效益的提升。高级品类收益和件型一致性收益的贡献显著，显示出本模型在仓储布局优化中的有效性。通过关注各个收益指标，企业可以在未来的仓储管理中更加精准地调整资源配置，以实现更高的经济效益和运营效率。

6.3 模型评估与分析

在模型评估与分析阶段，我们重点验证了模型的有效性、解的合理性和应用可行性。通过引入高级品类、件型一致性和品类关联度等收益项，模型量化了分仓方案的核心目标，并为这些收益赋予不同权重，确保优化方向符合实际业务需求。这种综合收益函数设计实现了收益的最大化，同时优化了整体分仓效益。

在成本控制与仓储约束方面，我们明确量化了总仓租成本，并限制每个品类的仓储数量，确保合理分配资源，避免仓储瓶颈。模拟退火算法的应用使得模型在多目标优化中具备灵活性，权重参数的调整增强了模型的适应性，使其能够更好地应对不同的业务需求。

此外，算法的收敛性与解的稳定性得到保障，模拟退火过程通过温度控制实现了解的稳定与合理，适用于大规模的分仓优化任务。模型输出的最佳分仓方案直观展示了仓库分配结果及综合收益值，为实际决策提供了有力支持，提升了仓储资源的利用效率。

尽管模型表现优异，但仍存在局限性，如对参数设置的敏感性和权重选择的依赖性。未来可以引入其他优化算法，或增加自适应权重调整机制，以更好适应动态的仓储需求。

6.4 分析不同方案中各业务指标的表现

不同方案主要指问题二中的一品一仓模式和问题三中的一品多仓模式。我们将根据题目描述的四类重要数据，即总成本、关联度收益、高级品类收益、件型一致性收益，作为业务指标进行衡量，以评估各自的表现和优势。

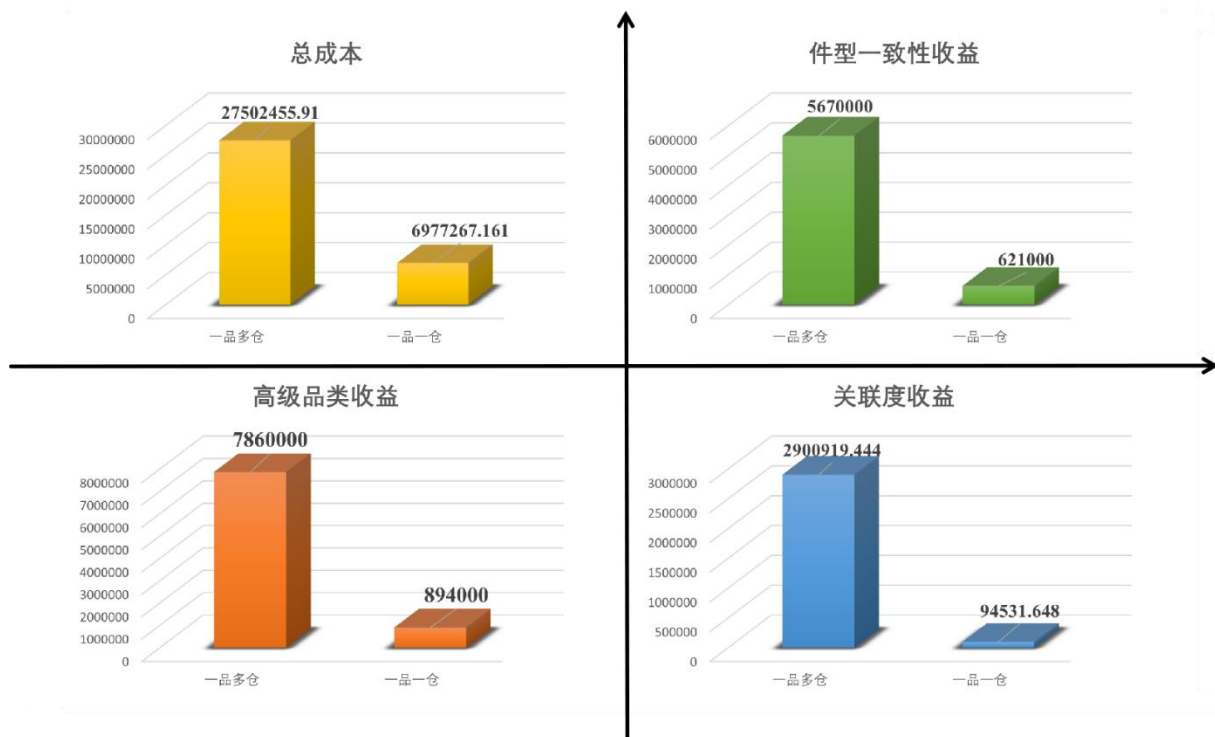


图 12 “一品一仓”和“一品多仓”的各项业务指标对比图

在本次模型求解中，对“一品多仓”和“一品一仓”两种模式下的四个关键指标进行了对比分析，包括总成本、关联度收益、高级品类收益和件型一致性收益。这些指标不仅反映了不同仓储模式的成本和收益差异，也为选择适合的仓储方案提供了数据支持。以下是对每个指标的详细分析。

总成本：一品多仓模式的总成本为 27,502,455.91 元，而一品一仓模式的总成本为 6,977,267.161 元，多仓模式的成本几乎是单仓模式的四倍。总成本是衡量仓储方案经济性的重要指标，它包括仓租成本、运营成本、物流成本等。多仓模式的高成本主要源于多个仓库的租赁和维护费用。虽然分仓可以降低部分运输距离，但在实际运营中，由于各个仓库之间的协调、运输和管理需求增加，导致总成本显著上升。管理多个仓库不仅需要更多的人力，还增加了操作复杂性，特别是在库存和订单管理方面，这进一步提升了运营成本。

在一品一仓模式中，集中管理一个仓库的成本较低，仓租和维护费用大幅减少，且由于无需多仓之间的调配，整体运输成本也更易控制。这种模式简化了管理流程，有助于降低成本，是适合较小市场或需求相对稳定的选择。然而，单仓模式在满足快速市场需求变化方面存在劣势，可能难以实现高效的区域配送。因此，从总成本的角度看，一品一仓具有显著的成本优势，但在需求分布广泛的市场中，其成本节省可能会受到配送时效的限制。

关联度收益：一品多仓模式的关联度收益为 390,167.564 元，而一品一仓模式为 94,531.648 元，前者的收益大约是后者的四倍。关联度收益反映了商品之间的交叉销售潜力，即将相关商品存放在同一仓库中，可以在履约过程中集中发货，减少包裹数量，降低物流成本，同时增加客户在一次购买中选购多种相关商品的可能性。多仓模式的高关联度收益表明，它能够更好地利用商品间的关系，优化仓储布局，实现更高效的履约。通过集中存放关联商品，一品多仓模式在交叉销售和降低配送成本方面更具优势。

相比之下，一品一仓模式的关联度收益较低，虽然可以通过集中管理简化仓储操作，但无法充分发挥商品关联度带来的价值。在单仓模式下，所有商品存放在同一地点，缺

少地理上的分散优势，导致无法利用区域市场需求的细微差异，影响了交叉销售机会。因此，虽然单仓模式能够减少管理复杂性，但在关联度收益方面处于劣势，难以充分挖掘商品间的协同效益。

高级品类收益：在高级品类收益上，一品多仓模式的收益为 7,860,000 元，而一品一仓模式为 894,000 元，多仓模式的收益是单仓模式的近十倍。高级品类收益是通过集中特定高价值商品而实现的收益提升。在一品多仓模式下，多个仓库能够覆盖更广泛的地理区域，使得高端商品更易接触到潜在高价值客户群体。分仓策略可以针对不同区域的消费特点实施差异化库存管理，提高高端商品的可获得性。此外，分仓策略还可以防止高端商品缺货，确保市场需求波动时仍能提供及时供应。

相较之下，一品一仓模式的高级品类收益较低，因其集中存储限制了高端商品的市场接触面。单仓模式在地理分布上缺乏灵活性，可能导致高端商品的物流时间延长，难以满足高端客户对服务效率的期望。这不仅影响客户满意度，也可能降低品牌形象。因此，一品一仓模式的高级品类收益远低于多仓模式，不利于企业在高端市场上的拓展和品牌建设。

件型一致性收益：一品多仓模式的件型一致性收益为 5,670,000 元，而一品一仓模式仅为 621,000 元，多仓模式在该收益项上高出单仓模式约 9 倍。件型一致性收益反映了相似件型商品在同一仓库中的存放效率。将同一件型的商品集中存放，能够提高包装和发货的便捷性，有助于减少订单错误率，提升发货效率。在一品多仓模式下，不同仓库可以根据件型特征进行商品分布，从而提升包装和配送效率。这种策略不仅减少了运输和包装过程中的错误，还可以显著降低运输成本。

而在一品一仓模式下，所有商品都集中在一个仓库，件型一致性的分布难以灵活调整，可能导致发货时的操作效率降低。特别是在订单量较大或商品种类繁多的情况下，单仓模式的作业压力增大，可能影响整体订单处理效率。因此，虽然一品一仓模式在管理复杂性上较低，但在实现件型一致性收益方面效果不如一品多仓。

综合分析这四个指标，一品一仓的优势在于成本低、管理简单，适合较小或需求稳定的市场。单仓模式可以集中资源、降低管理费用，并简化运输协调。然而，这一模式的灵活性不足，无法快速响应市场需求的变化，特别是在高端商品销售和关联商品的集中发货方面表现欠佳。

一品多仓模式则具有显著的市场覆盖和灵活性优势。通过多仓布局，企业能够更好地接触不同区域的消费者，尤其是高端客户群体，同时优化关联商品的集中存放，提升交叉销售的可能性。在件型一致性方面，多仓模式通过灵活的商品分布提升了包装和发货效率。然而，这一模式的缺点在于较高的运营成本和管理复杂性。多个仓库带来的成本压力较大，增加了协调难度，需要更成熟的管理机制和资源投入。

因此，企业在选择仓储模式时，需要综合考虑市场需求、产品特性、区域覆盖和管理能力。一品一仓适合成本敏感、管理资源有限的企业，而一品多仓更适合追求市场扩展、高效供应链和服务优化的企业。这种权衡有助于在降低成本和提升收益之间找到平衡点，确保仓储策略符合企业的长远发展目标。

七、模型分析检验

7.1 灵敏度分析

灵敏度分析有助于理解输入特征对各模型结果的影响。在库存和销量预测模型中，通过评估特征重要性和调整 SARIMA 模型参数，可以识别对预测结果影响较大的因素，帮助企业优化关键特征，从而提高预测的准确性。在仓库分配的线性规划模型中，灵敏度分析通过调整仓库容量、成本等参数，观察对最优分仓方案的影响，识别关键资源和

约束。模拟退火算法的灵敏度分析则关注温度衰减率和初始温度的设定，评估这些参数对收敛速度和解质量的影响。这些分析有助于优化模型参数，增强解的稳健性，使企业在面对市场波动时具备更好的应对能力。

7.2 误差分析

7.2.1 问题一的误差分析

问题一的误差主要来源于数据补全和模型预测。数据补全过程中，销量数据的缺失可能导致补全误差，尤其是在销售波动较大的时间点，如促销期间，补全后的数据可能未能真实反映实际情况。模型预测误差也可能出现，尤其对于波动频繁的品类，补全模型可能无法准确捕捉这些波动趋势。因此，问题一的预测结果虽然较为准确，但在高波动品类上可能仍存在一定误差。

7.2.2 问题二的误差分析

问题二中的线性规划模型假设仓库容量和需求为静态，未考虑实际变化带来的误差。此外，线性模型的简化可能忽略了非线性特征，使得模型与实际情况间存在一定的差异。同时，约束条件的松弛和求解时间限制可能影响解的精确性。数据的准确性对线性规划解的可靠性也有较大影响，如果问题一的数据预测存在误差，将进一步影响分仓方案的合理性。

7.2.3 问题三的误差分析

问题三使用模拟退火算法优化多仓分配方案。模拟退火引入的随机性会导致解的不稳定性，每次运行结果可能不同。此外，目标函数的简化在合并多个收益和成本项时可能忽略一些市场波动因素，影响实际解的效果。参数设定和数据准确性也是影响解质量的重要因素，若权重设置不准确，可能导致收益优化方向偏差。

7.3 整体误差分析

在整个问题求解中，误差来源主要来自数据补全、模型假设、算法随机性和参数设定。问题一的预测误差会影响后续问题的输入数据，从而在问题二和问题三中引发连锁误差。这表明模型的各部分紧密关联，强调了在模型构建和求解过程中保持数据准确性和模型灵活性的重要性，以减少误差对整体决策的影响。

八、 模型评价与推广

8.1 模型的优点

随机森林回归模型具有强大的非线性关系捕捉能力，能够处理复杂的特征交互，使得对多维数据的分析更加灵活和精准。它能有效处理非线性关系，减少过拟合的风险，并对异常值和缺失值具有较强的鲁棒性。这种模型通过集成多棵决策树，减少了过拟合的风险，同时提高了对新数据的预测准确性。

SARIMA 算法在时间序列数据建模方面表现出色，能够有效捕捉历史数据中的趋势和季节性变化。其自回归和移动平均的组合，使得 SARIMA 能够为时间序列数据提供稳健的预测，特别是在存在显著时间依赖性的情况下。因此，随机森林与 SARIMA 的结合，使得预测结果更为准确，并增强了对未来需求波动的适应能力。

采用线性规划模型优化仓库分配方案，这种方法的优势在于能够精确满足资源约束条件，如仓库的容量和产能限制。线性规划提供了一种系统化的决策支持工具，使得优化目标（如最小化仓租成本）能够在数学上严谨地表达。这一特性确保了分仓方案的合

理性和可操作性，为企业在资源配置上的决策提供了科学依据。通过线性规划，企业能够实现成本最小化或利润最大化，从而提升运营效率和竞争力。

模拟退火算法是一种基于随机搜索的优化技术，灵感来源于物理退火过程。其优点在于能够在复杂的解空间中寻找近似最优解，特别适合处理组合优化问题。通过在搜索过程中接受一定的劣解，模拟退火能够有效避免局部最优，从而提高全局搜索能力。在电商品类的多仓分配问题中，模拟退火算法可以灵活适应多种约束条件，快速找到有效的仓库分配方案，帮助企业在动态市场环境中优化资源配置，提升客户满意度和运营效率。

整体而言，将随机森林回归、SARIMA 算法、线性规划和模拟退火结合起来，可以形成一个综合性解决方案。这样的组合模型能够发挥各自的优势，处理复杂的预测与优化问题。例如，利用随机森林和 SARIMA 算法进行准确的销量和库存预测，再通过线性规划模型优化仓库分配，最后使用模拟退火算法进一步精细化解决方案。这种多层次、多维度的策略能够为企业全面的数据支持和决策依据，帮助其在竞争激烈的市场中保持领先地位。

8.2 模型的不足

在本次预测任务中，数据补全误差是需要特别关注的问题。实际应用中，商品库存量和销量的波动性和季节性可能也是重要的因素，由于部分品类的历史销量记录存在缺失，在补全这些缺失值的过程中可能引入误差，尤其是在销售趋势波动较大或数据不稳定的品类中。例如，在重大促销日（如“双十一”、“双十二”）或节假日，销量常出现短期急剧上升，这类突发波动通常难以通过常规的补全方法（如随机森林回归）精确还原。因此，缺失数据的补全往往只能基于整体趋势，而较难完全反映销售的突发变化，这使得在特定日期的销量可能被低估或高估。但本文未能考虑到这些因素的影响，一定程度上影响了模型的准确性。

由于第二问模型使用线性规划方法进行求解，模型中的目标函数和约束条件均为线性关系，这意味着模型假定仓租成本、容量、产能等指标与分配方案间呈线性关系。然而，实际仓租成本和产能往往具有非线性特征，例如产能的利用率可能呈现边际递减效果。线性近似导致的误差会影响模型输出的精确性，使得最终的分仓方案在实际应用中可能需要进一步调整。

8.3 模型的推广

本文提出的综合性解决方案在特定业务目标的实现上具备广泛的应用潜力。我们可以推广的应用场景有：

(1) 电商需求预测与库存管理：

针对电商行业，结合随机森林回归和 SARIMA 算法的预测能力，可以优化库存管理策略。企业可以根据历史销量数据及市场趋势，灵活调整库存水平，降低缺货和过剩的风险。在此基础上，利用线性规划优化仓库分配，确保资源高效利用。模拟退火算法可用于在动态市场环境中快速响应需求变化，进一步提升客户满意度。

(2) 制造业生产调度优化：

在制造业中，结合上述模型可以有效优化生产调度。通过随机森林和 SARIMA 算法分析生产需求，预测未来的原材料需求和产出量，随后应用线性规划模型进行生产资源的合理分配。模拟退火算法可以在多种生产约束条件下探索最优调度方案，确保生产效率的同时降低成本，从而提高整体运营效率。

参考文献

- [1] 赵广富,寇玮华,刘俊,等.基于 Prophet-SARIMA 模型的唐包线曹妃甸西站货运量预测研究[J].综合运输,2024,46(09):157-163.DOI:10.20164/j.cnki.cn11-1197/u.2024.09.030.
- [2] 李 景 . 电 商 销 量 预 测 及 其 分 仓 备 货 策 略 研 究 [D]. 南 京 大 学,2019.DOI:10.27235/d.cnki.gnjiu.2019.001925.
- [3] 王之威.电商物流分仓备货优化研究[D].北京交通大学,2016
- [4] 苏欣欣, 伊廷刚, 秦虎. 分支定价割平面法求解带时间窗和人力分配的车辆路径问题[J]. 交通运输工程与信息学报, 2021.
- [5] 董灵波.基于模拟退火算法的森林多目标经营规划模拟 [D]. 东北林业大学,2016.DOI:10.27009/d.cnki.gdblu.2016.000042.

附录

附录 1

第一问求解代码

```
import pandas as pd
from statsmodels.tsa.statespace.sarimax import SARIMAX

# 读取数据, 并将 '月份' 列转换为日期格式
data = pd.read_csv("D://下载/Integer_Interpolated_Inventory_Data_by_Category_and_Month.csv",
encoding='GB18030')
data['月份'] = pd.to_datetime(data['月份'], format='%Y/%m/%d')

# 计算每个品类的历史月均库存量
monthly_avg_inventory = data.groupby(['品类', data['月份'].dt.to_period('M')])['库存量'].mean().reset_index()
monthly_avg_inventory['月份'] = monthly_avg_inventory['月份'].dt.to_timestamp()

# 初始化一个字典用于存储预测结果
forecast_results = {'品类': [], '月份': [], '库存量': []}

# 遍历每个品类, 使用 SARIMA 模型进行预测
for category in monthly_avg_inventory['品类'].unique():
    # 提取该品类的月均库存数据, 并设置日期为索引
    category_data = monthly_avg_inventory[monthly_avg_inventory['品类'] == category].set_index('月份')['库存量']

    # 使用 SARIMA 模型拟合 (1, 1, 1)(1, 1, 1, 12) 模型
    try:
        model = SARIMAX(category_data, order=(1, 1, 1), seasonal_order=(1, 1, 1, 12))
        result = model.fit(dispatch=False)

        # 预测未来 3 个月的库存量 (7 月、8 月、9 月)
        forecast = result.get_forecast(steps=3).predicted_mean

        # 将预测结果保存到字典中
        for i, value in enumerate(forecast):
            month = f"{7 + i}月" # 对应 7 月、8 月、9 月
            forecast_results['品类'].append(category)
            forecast_results['月份'].append(month)
            forecast_results['库存量'].append(value)
    except Exception as e:
        print(f"品类 {category} 的模型拟合失败: {e}")

# 将预测结果转换为 DataFrame
forecast_df = pd.DataFrame(forecast_results)

# 保存结果为 CSV 文件, 编码为 GB18030, 与附件格式一致
forecast_df.to_csv("monthly_inventory_forecast3.csv", index=False, encoding='GB18030')

# 打印部分预测结果
print(forecast_df.head())
```

```
import pandas as pd
from statsmodels.tsa.statespace.sarimax import SARIMAX
from sklearn.model_selection import ParameterGrid

# 加载数据
```

```

data = pd.read_csv("D://下载/Integer_Interpolated_Inventory_Data_by_Category_and_Month.csv",
index_col=0)

# 确保日期列为 DateTime 格式并排序
data.columns = pd.to_datetime(data.columns, format="%Y/%m/%d")
data = data.sort_index(axis=1)

# SARIMA 参数搜索空间 (可根据实际情况调整范围)
param_grid = {
    'p': [0, 1, 2],
    'd': [0, 1],
    'q': [0, 1],
    'P': [0, 1],
    'D': [0, 1],
    'Q': [0, 1],
    's': [12] # 假设月度周期为 12
}

# 初始化数据框用于存储滚动预测数据
forecast_df = data.copy()

# 初始化数据框用于存储最优参数
optimal_params_df = pd.DataFrame(columns=['Category', 'p', 'd', 'q', 'P', 'D', 'Q', 's', 'AIC'])

# 为每个品类进行 SARIMA 参数优化和滚动预测
for category in data.index:
    print(f'正在为品类 '{category}' 优化参数并预测...')

    # 准备当前品类的时间序列数据
    ts = data.loc[category].dropna()

    # 定义变量以存储最优模型和最小的 AIC 值
    best_aic = float("inf")
    best_params = None

    # 网格搜索, 遍历所有参数组合
    for params in ParameterGrid(param_grid):
        try:
            # 尝试拟合 SARIMA 模型
            model = SARIMAX(ts,
                            order=(params['p'], params['d'], params['q']),
                            seasonal_order=(params['P'], params['D'], params['Q'], params['s']),
                            enforce_stationarity=False, enforce_invertibility=False)

            results = model.fit(dispatch=False)

            # 比较 AIC 值, 保留最优参数组合
            if results.aic < best_aic:
                best_aic = results.aic
                best_params = params
        except Exception as e:
            print(f'在品类 '{category}' 的参数 {params} 下出错: {e}')
            continue

    # 保存最优参数到数据框
    if best_params is not None:
        optimal_params_df = pd.concat([optimal_params_df, pd.DataFrame([{'Category': category,
                                'p': best_params['p'],
                                'd': best_params['d'],
                                'q': best_params['q'],
                                'P': best_params['P'],
                                'D': best_params['D'],
                                'Q': best_params['Q'],
                                's': best_params['s'],
                                'AIC': best_aic}])])

```

```

        'D': best_params['D'],
        'Q': best_params['Q'],
        's': best_params['s'],
        'AIC': best_aic
    }]], ignore_index=True)

# 使用最优参数进行滚动预测
print(f'品类 '{category}' 的最优参数: {best_params}, AIC: {best_aic}')

rolling_ts = ts.copy()

for i in range(3): # 预测未来 3 个月
    try:
        # 使用最优参数拟合模型
        model = SARIMAX(rolling_ts,
                        order=(best_params['p'], best_params['d'], best_params['q']),
                        seasonal_order=(best_params['P'], best_params['D'], best_params['Q'],
best_params['s']),
                        enforce_stationarity=False, enforce_invertibility=False)
        results = model.fit(dispatch=False)

        # 预测下一个月
        forecast = results.get_forecast(steps=1)
        forecast_value = forecast.predicted_mean.iloc[0].round().astype(int) # 取整后的预测值
        forecast_date = rolling_ts.index[-1] + pd.DateOffset(months=1) # 新的预测日期

        # 将预测结果添加到滚动时间序列和最终的数据框中
        rolling_ts[forecast_date] = forecast_value
        forecast_df.loc[category, forecast_date] = forecast_value
    except Exception as e:
        print(f'在品类 '{category}' 的第 {i + 1} 个月预测时出错: {e}')
        break

# 将所有日期列格式化为字符串，以匹配原始输入格式
forecast_df.columns = forecast_df.columns.strftime("%Y/%m/%d")

# 输出预测结果到 CSV 文件
forecast_df.to_csv('优化滚动预测结果_未来三个月.csv', encoding='utf-8-sig')
# 输出最优参数到 CSV 文件
optimal_params_df.to_csv('最优参数结果.csv', encoding='utf-8-sig', index=False)

print("滚动预测结果已保存到 '优化滚动预测结果_未来三个月.csv'")
print("最优参数结果已保存到 '最优参数结果.csv'")

```

```

import pandas as pd
import numpy as np
from statsmodels.tsa.statespace.sarimax import SARIMAX
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt

# Step 1: 数据加载与预处理
# 假设数据为一个 CSV 文件，其中 index 为月份，列为各品类库存量
data = pd.read_csv("C://Users/ALAN/Desktop/附件/附件 1.csv", encoding='GB18030')

# Step 2: SARIMA 模型预测

```

```

def sarima_forecast(series, order=(1, 1, 1), seasonal_order=(1, 1, 1, 12), steps=3):
    model = SARIMAX(series, order=order, seasonal_order=seasonal_order, enforce_stationarity=False,
                    enforce_invertibility=False)
    sarima_fit = model.fit(dispatch=False)
    forecast = sarima_fit.forecast(steps=steps)
    return forecast

# Step 3: LSTM 模型预测
# 数据标准化 - 仅对第三列进行归一化
scaler = MinMaxScaler(feature_range=(0, 1))
data.iloc[:, 2] = scaler.fit_transform(data.iloc[:, 2].values.reshape(-1, 1))

# 将第三列的归一化结果提取为 `scaled_data`, 用于后续建模
scaled_data = data.iloc[:, 2].values.reshape(-1, 1) # `scaled_data` 仅为第三列的归一化数据

# 构建 LSTM 输入数据集
def create_lstm_dataset(dataset, time_step=12):
    X, Y = [], []
    for i in range(len(dataset) - time_step - 1):
        X.append(dataset[i:(i + time_step), 0])
        Y.append(dataset[i + time_step, 0])
    return np.array(X), np.array(Y)

time_step = 12
X, Y = create_lstm_dataset(scaled_data)
X = X.reshape(X.shape[0], X.shape[1], 1)

# 构建 LSTM 模型
lstm_model = Sequential()
lstm_model.add(LSTM(50, return_sequences=True, input_shape=(time_step, 1)))
lstm_model.add(LSTM(50, return_sequences=False))
lstm_model.add(Dense(25))
lstm_model.add(Dense(1))
lstm_model.compile(optimizer='adam', loss='mean_squared_error')
lstm_model.fit(X, Y, epochs=10, batch_size=1, verbose=2)

# Step 4: 预测与融合
sarima_forecasts = []
lstm_forecasts = []

for category in data.columns:
    # SARIMA 预测
    sarima_pred = sarima_forecast(data[category])
    sarima_forecasts.append(sarima_pred)

    # LSTM 预测
    lstm_input = scaled_data[-time_step:].reshape(1, time_step, 1)
    lstm_pred_scaled = lstm_model.predict(lstm_input)
    lstm_pred = scaler.inverse_transform(lstm_pred_scaled)
    lstm_forecasts.append(lstm_pred.flatten())

# 融合预测 (简单加权平均)
alpha = 0.65745
combined_forecasts = alpha * np.array(sarima_forecasts) + (1 - alpha) * np.array(lstm_forecasts)

# Step 5: 输出预测结果
forecast_df = pd.DataFrame(combined_forecasts, columns=['7 月库存量', '8 月库存量', '9 月库存量'],
                           index=data.columns)

```

```
print(forecast_df)
forecast_df.to_csv('monthly_inventory_forecast3.csv')
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller

# 加载数据
data = pd.read_csv("D://下载/Integer_Interpolated_Inventory_Data_by_Category_and_Month.csv",
index_col=0)

# 确保日期列为 DateTime 格式并排序
data.columns = pd.to_datetime(data.columns, format='%Y/%m/%d')
data = data.sort_index(axis=1)

# 计算每个品类的标准差（波动性）
std_devs = data.std(axis=1)

# 选择标准差排名前 5 的品类（波动性较大）
top_volatility_categories = std_devs.nlargest(5).index

# 定义差分并检查平稳性的函数
def difference_until_stationary(series, max_diffs=5):
    """差分直至序列平稳，最多进行 max_diffs 次差分"""
    diff_series = series
    for i in range(max_diffs):
        result = adfuller(diff_series)
        p_value = result[1]
        if p_value < 0.05: # 如果 p 值小于 0.05，则认为序列平稳
            print(f'Stationary after {i} differences with p-value: {p_value:.4f}')
            return diff_series, i # 返回平稳序列和差分次数
        diff_series = diff_series.diff().dropna() # 进行一阶差分
    print(f'Max differences reached without stationarity. Last p-value: {p_value:.4f}')
    return diff_series, max_diffs

# 进行差分处理直至平稳
stationary_data = {}
for category in top_volatility_categories:
    diff_series, num_diffs = difference_until_stationary(data.loc[category])
    stationary_data[category] = {
        'Differenced Data': diff_series,
        'Num of Differences': num_diffs
    }

# 可视化差分后的结果
fig, axs = plt.subplots(nrows=len(top_volatility_categories), ncols=1, figsize=(12, len(top_volatility_categories)
* 4))

for idx, category in enumerate(top_volatility_categories):
    axs[idx].plot(stationary_data[category]['Differenced Data'].index, stationary_data[category]['Differenced
Data'], label=category)
    # axs[idx].set_title(f'Differenced Data for {category} (Differences: {stationary_data[category]["Num of
Differences"]})')
    axs[idx].set_ylabel('Differenced Value')
    axs[idx].grid()
    axs[idx].legend(loc='best')

# 调整子图之间的间距
```

```
plt.subplots_adjust(hspace=2) # 可以根据需要调整这个值
```

```
plt.tight_layout()
plt.show()
```

```
import matplotlib.pyplot as plt
import pandas as pd
#
# # Convert the "月份" column to datetime format
# forecast_df['月份'] = pd.to_datetime(forecast_df['月份'], format='%Y-%m')
#
# # List unique categories for visualization
# categories = forecast_df['品类'].unique()
#
# # Plot the forecasted inventory levels for each category
# plt.figure(figsize=(12, 8))
#
# for category in categories[:10]: # Limiting to first 10 categories for clarity in visualization
#     category_data = forecast_df[forecast_df['品类'] == category]
#     plt.plot(category_data['月份'], category_data['预测库存量'], label=category)
#
# # Add title and labels
# plt.title('Inventory Forecasts for Selected Categories (First 10)')
# plt.xlabel('Month')
# plt.ylabel('Forecasted Inventory Level')
# plt.legend(title="Category", loc='upper left', bbox_to_anchor=(1, 1))
# plt.xticks(rotation=45)
# plt.tight_layout()
# plt.show()

import pandas as pd
import matplotlib.pyplot as plt

# Load the uploaded ADF results file
adf_results_file_path = "D://下载/Updated_ADF_Results_with_Stationarity.csv"
adf_results_df = pd.read_csv(adf_results_file_path, index_col=0)

# Remove 'category' prefix, keeping only the numeric part of the category names
adf_results_df.index = adf_results_df.index.str.extract('(\\d+)')[0].astype(int)

# Sort the DataFrame by p-value for better visualization
adf_results_df_sorted = adf_results_df.sort_values(by='ADF_p_value')

# Sample every 10th category for visualization
sampled_adf_results = adf_results_df_sorted.iloc[::5]

# Plotting the sampled ADF p-values
plt.figure(figsize=(10, 6))
plt.bar(sampled_adf_results.index.astype(str), sampled_adf_results['ADF_p_value'], color='lightblue')
plt.axhline(y=0.05, color='red', linestyle='--', label='Stationarity Threshold (p = 0.05)')

# Adding labels and title
plt.xlabel('Category (Numeric)', fontsize=12)
plt.ylabel('ADF p-value', fontsize=12)
plt.title('Sampled ADF Test Results (Numeric Categories)', fontsize=14)
plt.xticks(rotation=90)
plt.legend()

# Display the plot
plt.tight_layout()
plt.show()
```

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.statespace.sarimax import SARIMAX
from sklearn.model_selection import ParameterGrid

# 加载数据
data = pd.read_csv("D://下载/Integer_Interpolated_Inventory_Data_by_Category_and_Month.csv",
index_col=0)

# 确保日期列为 DateTime 格式并排序
data.columns = pd.to_datetime(data.columns, format='%Y/%m/%d')
data = data.sort_index(axis=1)

# 计算每个品类的标准差（波动性）
std_devs = data.std(axis=1)

# 选择标准差排名前 20 的品类（波动性较大）
top_volatility_categories = std_devs.nlargest(10).index

# 进行一阶差分
diff_data = data.loc[top_volatility_categories].diff(axis=1).dropna(axis=1) # 在列方向上进行差分并去除缺失值

# 输出波动性较大的品类
print("Top Volatility Categories:")
print(top_volatility_categories)

# 对差分后的数据进行 ADF 检验
def adf_test(series):
    result = adfuller(series)
    return result[0], result[1], result[4] # 返回 ADF 统计量、p 值和临界值

# 检验每个差分后的品类并保存结果
adf_results = {}

for category in top_volatility_categories:
    adf_stat, p_value, critical_values = adf_test(diff_data.loc[category])
    adf_results[category] = {
        'ADF Statistic': adf_stat,
        'p-value': p_value,
        'Critical Values': critical_values
    }

# 打印 ADF 检验结果
print("\nADF Test Results for Differenced Data:")
for category, results in adf_results.items():
    print(f"\nCategory: {category}")
    print(f"    ADF Statistic: {results['ADF Statistic']}")
    print(f"    p-value: {results['p-value']}")
    print(f"    Critical Values:")
    for key, value in results['Critical Values'].items():
        print(f"        {key}: {value}")

# 可视化差分数据
fig, axs = plt.subplots(nrows=1, ncols=1, figsize=(12, 6))

```

```

for category in top_volatility_categories:
    axs.plot(diff_data.columns, diff_data.loc[category], label=category)

# 添加 ADF 结果的文本说明
for idx, category in enumerate(top_volatility_categories):
    adf_stat = adf_results[category]['ADF Statistic']
    p_value = adf_results[category]['p-value']
    axs.text(0.05, 0.95 - idx * 0.05,
             f'{category}: ADF Stat={adf_stat:.2f}, p-value={p_value:.4f}',
             transform=axs.transAxes, fontsize=10)

axs.set_title('Differenced Data of Top 20 Volatile Inventory Categories')
axs.set_xlabel('Date')
axs.set_ylabel('Differenced Value')
axs.legend(loc='center left', bbox_to_anchor=(1, 0.5))
axs.grid()
axs.tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()

```

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Reload the CSV file
file_path = "D://下载/Reshaped_Inventory_Data_by_Category_and_Month.csv"
try:
    data = pd.read_csv(file_path, encoding='GBK')
except UnicodeDecodeError:
    # If 'ISO-8859-1' doesn't work, try 'gbk'
    data = pd.read_csv(file_path, encoding='gbk')

# Set the '品类' column as the index and exclude it from the columns list
data.set_index('品类', inplace=True)

# Convert the columns to datetime format
data.columns = pd.to_datetime(data.columns)

# Transpose the dataframe to have dates as rows and categories as columns
data = data.T

# Calculate the fluctuation (range) and mean for each category
category_fluctuations = data.max() - data.min()
category_means = data.mean()

# Define threshold for selecting similar magnitudes, e.g., within a 20% range around the median
magnitude_threshold = 0.2 * category_means.median()

# Filter categories by similar mean values around the median and with noticeable fluctuations
filtered_categories = category_means[
    (category_means >= (category_means.median() - magnitude_threshold)) &
    (category_means <= (category_means.median() + magnitude_threshold))
].index

# Further filter these categories to get the top fluctuating ones
num_categories_to_plot = 20
top_fluctuating_similar_categories =
category_fluctuations[filtered_categories].nlargest(num_categories_to_plot).index

# Plot the time series for the selected categories
plt.figure(figsize=(18, 12))

```

```

for column in top_fluctuating_similar_categories:
    plt.plot(data.index, data[column], marker='o', label=column)

plt.title('Time series plot: High-fluctuation categories with similar magnitudes')
plt.xlabel('Months')
plt.ylabel('Predicted Value')
plt.grid(True)
plt.tight_layout(rect=[0, 0, 0.85, 1])

# Place the legend outside the plot
plt.legend(loc='center left', bbox_to_anchor=(1, 0.5), ncol=1)
plt.show()

```

```

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.spatial.distance import squareform
from fastdtw import fastdtw # 高效实现 DTW
from scipy.spatial.distance import euclidean

# 1. 从文件读取数据
file_path = "D://下载/Reshaped_Inventory_Data_by_Category_and_Month.csv"
data = pd.read_csv(file_path, index_col=0, encoding='GBK')

# 2. 数据预处理：确保日期为 DateTime 格式并排序
data.columns = pd.to_datetime(data.columns, format='%Y/%m/%d')
data = data.sort_index(axis=1) # 按列（时间）排序

# 3. 初始化 DTW 距离矩阵
num_products = data.shape[0] # 商品数量（行数）
dtw_matrix = np.zeros((num_products, num_products)) # 初始化空矩阵

# 4. 计算每对商品之间的 DTW 距离
for i in range(num_products):
    for j in range(i + 1, num_products):
        # 使用 fastdtw 计算两时间序列之间的 DTW 距离
        distance, _ = fastdtw(data.iloc[i], data.iloc[j], dist=euclidean)
        dtw_matrix[i, j] = distance
        dtw_matrix[j, i] = distance # 矩阵是对称的

# 5. 绘制 DTW 距离矩阵的热力图
plt.figure(figsize=(20, 10))
sns.heatmap(dtw_matrix, cmap='YlGnBu', xticklabels=False, yticklabels=False)
plt.title('DTW Distance Matrix of Product Inventory', fontsize=16)
plt.xlabel('Products', fontsize=12)
plt.ylabel('Products', fontsize=12)
plt.colorbar(label='DTW Distance')
plt.show()

```

```

import pandas as pd
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# 1. 从文件读取数据
file_path = "D://下载/Reshaped_Inventory_Data_by_Category_and_Month.csv" # 输入文件路径
data = pd.read_csv(file_path, index_col=0, encoding='GBK')

# 2. 数据预处理：确保列为日期格式并排序

```

```

data.columns = pd.to_datetime(data.columns, format='%Y/%m/%d')
data = data.sort_index(axis=1)

# 3. 汇总数据的描述性统计特征
extracted_features = data.describe().T # 按列（时间）提取统计特征

# 4. 标准化数据
scaler = StandardScaler()
data_scaled = scaler.fit_transform(extracted_features)

# 5. PCA 降维：提取前两个主成分
pca = PCA(n_components=2)
pca_result = pca.fit_transform(data_scaled)

# 6. 获取解释的方差比例
explained_variance_ratio = pca.explained_variance_ratio_

# 7. 绘制 PCA 结果并显示解释的方差比例
plt.figure(figsize=(8, 6))
plt.scatter(pca_result[:, 0], pca_result[:, 1], alpha=0.6, edgecolors='k')

# 将方差比例显示在图中（注释）
explained_text = (f'Explained variance:\n'
                  f'PC1: {explained_variance_ratio[0]:.2%}\n'
                  f'PC2: {explained_variance_ratio[1]:.2%}')

# 将解释的方差比例显示在图的右上角
plt.annotate(explained_text, xy=(0.95, 0.95), xycoords='axes fraction',
             fontsize=10, ha='right', va='top', bbox=dict(boxstyle='round,pad=0.5', fc='lightgrey', alpha=0.6))

# 设置标题和坐标轴标签
plt.title('PCA of Categories')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.grid(True)

# 显示图像
plt.show()

# 8. 输出主成分解的方差比例
print("Explained variance ratio:", explained_variance_ratio)

```

```

import pandas as pd
from statsmodels.tsa.statespace.sarimax import SARIMAX
from sklearn.metrics import mean_squared_error
import itertools

# 加载数据
data = pd.read_csv("D://下载/Integer_Interpolated_Inventory_Data_by_Category_and_Month.csv",
                  index_col=0)

# 确保日期列为 DateTime 格式并排序
data.columns = pd.to_datetime(data.columns, format='%Y/%m/%d')
data = data.sort_index(axis=1)
best_params_dict = {}
# 选择要优化的参数范围
# 缩小参数范围
p = range(0, 2)
d = range(0, 2)
q = range(0, 2)
P = range(0, 2)

```

```

D = range(0, 2)
Q = range(0, 2)
s = [12]
# 生成所有参数组合
parameters = list(itertools.product(p, d, q, P, D, Q, s))

# 使用 2022 年 7 月、8 月和 9 月的库存数据作为验证集
validation_data = data.loc[:, pd.to_datetime(['2022-07-01', '2022-08-01', '2022-09-01'])]

# 初始化变量以存储最佳结果
best_score, best_params = float("inf"), None

# 遍历每个品类进行参数优化
for category in data.index:
    ts = data.loc[category].dropna()

    for param in parameters:
        try:
            # 设置 SARIMA 模型
            model = SARIMAX(ts, order=(param[0], param[1], param[2]),
                            seasonal_order=(param[3], param[4], param[5], param[6]),
                            enforce_stationarity=False, enforce_invertibility=False)
            results = model.fit(dispatch=False)

            # 预测 2022 年 7-9 月
            forecast = results.get_forecast(steps=3)
            forecast_values = forecast.predicted_mean

            # 计算验证集上的误差（使用均方误差）
            mse = mean_squared_error(validation_data.loc[category], forecast_values)

            # 检查是否为最佳模型
            if mse < best_score:
                best_score, best_params = mse, param
                best_params_dict[category] = {'params': best_params, 'mse': best_score}
                print(f'品类: {category}, 新最佳参数: {best_params}, MSE: {best_score}')

        except Exception as e:
            print(f'在品类 '{category}' 上使用参数 {param} 时出错: {e}')

print(f'最佳参数组合为: {best_params}, 对应的均方误差为: {best_score}')
# 转换为数据框并保存到 CSV
best_params_df = pd.DataFrame(best_params_dict).T
best_params_df.to_csv("最佳参数组合.csv", encoding='utf-8-sig')

print("每个品类的最佳参数组合已保存到 '最佳参数组合.csv'")

```

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split

# 1. 读取 Excel 文件中的数据（替换为您的文件路径）
file_path = r"C:\Users\马瑞波\Desktop\附件\附件二\batch_5_categories.xlsx" # 使用用户上传的路径
data = pd.read_excel(file_path)

# 2. 获取所有品类的列表
categories = data['品类'].unique()

```

```

# 3. 创建一个用于存储所有品类预测结果的列表
all_predictions = []

# 4. 遍历每个品类并进行预测
for category in categories:
    print(f'Processing category: {category}')

    # 筛选当前品类的数据
    category_data = data[data['品类'] == category]

    # 按日期聚合销量数据
    category_sales = category_data.groupby('日期')['销量'].sum().reset_index()

    # 创建日期相关特征和滞后特征
    category_sales['day'] = category_sales['日期'].dt.day
    category_sales['month'] = category_sales['日期'].dt.month
    category_sales['weekday'] = category_sales['日期'].dt.weekday

    # 生成滞后特征
    for lag in [1, 7, 14, 30]:
        category_sales[f'lag_{lag}'] = category_sales['销量'].shift(lag)

    # 去除缺失值
    category_sales = category_sales.dropna()

    # 划分特征和目标变量
    X = category_sales[['day', 'month', 'weekday', 'lag_1', 'lag_7', 'lag_14', 'lag_30']]
    y = category_sales['销量']

    # 划分训练集和测试集
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)

    # 初始化并训练随机森林模型
    rf_model = RandomForestRegressor(n_estimators=100, max_depth=10, random_state=42)
    rf_model.fit(X_train, y_train)

    # 使用最后 30 天的数据生成滞后特征以进行未来预测
    last_days = category_sales.tail(30).copy()
    predictions = []

    # 逐步预测未来 3 个月（7 月 1 日至 9 月 30 日）
    for date in pd.date_range('2023-07-01', '2023-09-30'):
        day = date.day
        month = date.month
        weekday = date.weekday()

        # 生成滞后特征
        lag_1 = last_days['销量'].iloc[-1]
        lag_7 = last_days['销量'].iloc[-7] if len(last_days) >= 7 else 0
        lag_14 = last_days['销量'].iloc[-14] if len(last_days) >= 14 else 0
        lag_30 = last_days['销量'].iloc[-30] if len(last_days) >= 30 else 0

        # 创建当前日期的输入特征
        X_input = pd.DataFrame([[day, month, weekday, lag_1, lag_7, lag_14, lag_30]],
                               columns=['day', 'month', 'weekday', 'lag_1', 'lag_7', 'lag_14', 'lag_30'])

        # 使用模型预测
        predicted_sales = rf_model.predict(X_input)[0]

        # 存储预测结果

```

```

        predictions.append({'date': date, 'category': category, 'predicted_sales': predicted_sales})

    # 更新 last_days
    last_days = pd.concat([last_days, pd.DataFrame({'日期': [date], '销量': [predicted_sales]})])

    # 将预测结果存入 DataFrame
    future_data = pd.DataFrame(predictions)

    # 合并历史与预测数据
    category_sales['品类'] = category # 添加品类列
    category_sales['type'] = 'Historical'
    category_sales.rename(columns={'销量': 'sales'}, inplace=True)

    future_data['type'] = 'Forecast'

    all_data = pd.concat([category_sales[['日期', '品类', 'sales', 'type']],
                          future_data[['date', 'category', 'predicted_sales', 'type']].rename(columns={'date':
    '日期', 'predicted_sales': 'sales'})])

    # 存储每个品类的预测结果
    all_predictions.append(all_data)

    # 绘制折线图
    plt.figure(figsize=(18, 9))
    for data_type, subset in all_data.groupby('type'):
        plt.plot(subset['日期'], subset['sales'], marker='o', linestyle='-', markersize=4, label=data_type)

    plt.title(f'{category} - Historical and Forecast Sales Line Chart', fontsize=16)
    plt.xlabel('Date', fontsize=14)
    plt.ylabel('Sales', fontsize=14)
    plt.legend(fontsize=12)
    plt.grid(True)
    plt.xticks(rotation=45)
    plt.show()

```

```

# 导入必要的库
import pandas as pd
import matplotlib.pyplot as plt

# 读取 CSV 文件并处理编码问题
file_path = r"F:\数模\dashujubei\2024 年 MathorCup 大数据挑战赛-初赛赛题\2024 年 MathorCup 大数据挑战
赛-赛道 B 初赛\附件\附件 2.csv" # 根据您的路径进行修改
data = pd.read_csv(file_path, encoding='ISO-8859-1')

# 重命名列，便于理解
data.columns = ['Category', 'Date', 'Sales']

# 将 'Date' 列转换为日期格式
data['Date'] = pd.to_datetime(data['Date'], errors='coerce')

# 统计每个品类出现的唯一天数
category_days_count = data.groupby('Category')['Date'].nunique().reset_index()
category_days_count.columns = ['Category', 'Unique_Days']

# 提取品类中的数字部分作为标签
category_days_count['Category_Label'] = category_days_count['Category'].str.extract('(\\d+)').astype(int)

```

```

# 绘制散点图
plt.figure(figsize=(12, 6))
plt.scatter(category_days_count['Category_Label'], category_days_count['Unique_Days'], alpha=0.7)
plt.xticks(rotation=90) # 横坐标标签旋转 90 度, 防止重叠
plt.xlabel('Category (Numeric Label)')
plt.ylabel('Number of Unique Days')
plt.title('Number of Days Each Category Appears') # 确保这里拼写无误
plt.grid(True) # 添加网格线

# 展示图表
plt.show()

```

```

import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor

# 1. 读取 Excel 文件中的数据 (替换为您的文件路径)
file_path = r"C:\Users\马瑞波\Desktop\附件\附件 2.xlsx" # 替换为实际路径
data = pd.read_excel(file_path)

# 2. 创建所需的完整日期范围 (2022-07-01 到 2022-09-30 和 2023-04-01 到 2023-06-30)
date_range_1 = pd.date_range('2022-07-01', '2022-09-30')
date_range_2 = pd.date_range('2023-04-01', '2023-06-30')
full_date_range = date_range_1.union(date_range_2)

# 3. 创建一个用于存储所有品类完整数据的列表
complete_data = []

# 4. 遍历每个品类进行数据检测与补全
categories = data['品类'].unique()

for category in categories:
    print(f'正在处理品类: {category}')

    # 筛选当前品类的数据并与完整日期范围对齐
    category_data = data[data['品类'] == category].set_index('日期')
    category_data = category_data.reindex(full_date_range, fill_value=np.nan)
    category_data['品类'] = category # 补回品类列

    # 检查是否有缺失值
    missing_dates = category_data['销量'].isna().sum()
    if missing_dates > 0:
        print(f'品类 {category} 缺失 {missing_dates} 天数据, 正在进行补全...')

    # 填充已有数据的日期特征
    category_data['day'] = category_data.index.day
    category_data['month'] = category_data.index.month
    category_data['weekday'] = category_data.index.weekday

    # 分离已知数据和需要预测的数据
    known_data = category_data.dropna()
    unknown_data = category_data[category_data['销量'].isna()]

    # 准备特征和目标变量用于模型训练
    X_train = known_data[['day', 'month', 'weekday']]
    y_train = known_data['销量']

    # 初始化并训练随机森林模型
    rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
    rf_model.fit(X_train, y_train)

```

```

# 使用模型对缺失值进行预测
X_unknown = unknown_data[['day', 'month', 'weekday']]
predicted_sales = rf_model.predict(X_unknown)

# 将预测结果填充到原数据中
category_data.loc[category_data['销量'].isna(), '销量'] = predicted_sales

# 将完整的数据添加到列表中
complete_data.append(category_data.reset_index().rename(columns={'index': '日期'}))

# 5. 合并所有品类的数据
complete_data_df = pd.concat(complete_data)

# 6. 输出完整的历史数据为 CSV 文件

print(f'所有品类的历史数据已补全并保存为 。')

```

```

# 姓名：马瑞波
# 时间：2024/10/26 10:30
import pandas as pd

# 1. 读取预处理后的 Excel 文件（替换为您的文件路径）
file_path = r"C:\Users\马瑞波\Desktop\附件\预处理后的附件二.xlsx" # 替换为实际路径
data = pd.read_excel(file_path)

# 2. 获取所有品类的列表
categories = data['品类'].unique()

# 3. 设置批次大小和总批次数量
batch_size = 70 # 每批处理 70 个品类
total_batches = len(categories) // batch_size + (1 if len(categories) % batch_size != 0 else 0)

# 4. 遍历每个批次并保存为单独的 Excel 文件
for i in range(total_batches):
    # 获取当前批次的品类
    batch_categories = categories[i * batch_size: (i + 1) * batch_size]

    # 筛选当前批次的数据
    batch_data = data[data['品类'].isin(batch_categories)]

    # 构建保存文件的路径和名称
    batch_file_path = f"batch_{i + 1}_categories.xlsx"

    # 保存为 Excel 文件
    batch_data.to_excel(batch_file_path, index=False)

    print(f'批次 {i + 1} 已保存为 {batch_file_path}')

```

第二问求解代码

```

import pandas as pd
from pulp import LpProblem, LpMinimize, LpVariable, lpSum, PULP_CBC_CMD

# 假设文件已正确加载到以下变量
forecast_data = pd.read_excel(r"C:\Users\马瑞波\Desktop\附件\所有品类预测结果.xlsx")
sales_forecast = pd.read_excel(r"C:\Users\马瑞波\Desktop\附件\问题一的结果.xlsx") # 假设文件包含日销量数据

```

```

# 处理库存预测数据
inventory_data = forecast_data.pivot(index='品类', columns='月份', values='预测库存量 1 (sarima)')
inventory_data['月库存量'] = inventory_data[['7月', '8月', '9月']].max(axis=1)

# 处理销量数据
sales_data = sales_forecast.set_index('品类').groupby(level=0).sum() # 每月总销量
sales_data['月销量'] = sales_data.max(axis=1) # 取最大月销量

# 读取并处理仓库信息
warehouse_info = pd.read_excel(r"C:\Users\马瑞波\Desktop\附件\附件 3.xlsx") # 假设已转成xlsx格式
warehouse_info['月仓容上限'] = warehouse_info['仓容上限'] * 30
warehouse_info['月产能上限'] = warehouse_info['产能上限'] * 30
warehouse_info['月租成本'] = warehouse_info['仓租日成本'] * 30

# 构建规划模型
categories = inventory_data.index.tolist()
warehouses = warehouse_info['仓库'].tolist()
model = LpProblem("Category-Warehouse-Allocation", LpMinimize)

# 决策变量
allocation = LpVariable.dicts("Allocation", ((cat, wh) for cat in categories for wh in warehouses), cat='Binary')

# 目标函数
warehouse_rent = dict(zip(warehouse_info['仓库'], warehouse_info['月租成本']))
model += lpSum(allocation[cat, wh] * warehouse_rent[wh] for cat in categories for wh in warehouses)

# 约束 1: 一品一仓
for cat in categories:
    model += lpSum(allocation[cat, wh] for wh in warehouses) == 1

# 约束 2: 仓容上限
warehouse_capacity = dict(zip(warehouse_info['仓库'], warehouse_info['月仓容上限']))
for wh in warehouses:
    model += lpSum(allocation[cat, wh] * inventory_data.loc[cat, '月库存量'] for cat in categories) <= warehouse_capacity[wh]

# 约束 3: 产能上限
warehouse_output_limit = dict(zip(warehouse_info['仓库'], warehouse_info['月产能上限']))
for wh in warehouses:
    model += lpSum(allocation[cat, wh] * sales_data.loc[cat, '月销量'] for cat in categories) <= warehouse_output_limit[wh]

# 设置求解器参数, 增加求解时间
solver = PULP_CBC_CMD(msg=True, timeLimit=600) # 设置最大求解时间为 300 秒
model.solve(solver)

# 输出局部最优解
allocation_results = {(cat, wh): allocation[cat, wh].varValue for cat in categories for wh in warehouses if allocation[cat, wh].varValue == 1}
allocation_df = pd.DataFrame(allocation_results.keys(), columns=['品类', '仓库'])
allocation_df.to_excel('分仓方案.xlsx', index=False)

# 输出局部最优解的信息
print("局部最优解: ")
print(allocation_df)
print("最优目标值: ", model.objective.value())

```

第三问的求解代码

```
import random
import numpy as np
import pandas as pd
from collections import defaultdict
import os

# 加载数据
file_path1 = r"C:\Users\马瑞波\Desktop\附件\附件 3.xlsx" # 仓库信息
file_path2 = r"C:\Users\马瑞波\Desktop\附件\附件 4.xlsx" # 关联度信息
file_path3 = r"C:\Users\马瑞波\Desktop\附件\问题一的结果.xlsx" # 预测日销量
file_path4 = r"C:\Users\马瑞波\Desktop\附件\所有品类预测结果.xlsx" # 预测月库存量
file_path5 = r"C:\Users\马瑞波\Desktop\附件\附件 5.xlsx" # 件型和高级品类信息

# 读取数据
warehouse_data = pd.read_excel(file_path1)
correlation_data = pd.read_excel(file_path2)
daily_sale_data = pd.read_excel(file_path3)
monthly_store_data = pd.read_excel(file_path4)
category_data = pd.read_excel(file_path5)

# 将仓库上限、产能上限和仓租日成本转换为月单位
warehouse_data['仓租月成本'] = warehouse_data['仓租日成本'] * 30
warehouse_data['仓库上限 (月)'] = warehouse_data['仓库上限'] * 30
warehouse_data['产能上限 (月)'] = warehouse_data['产能上限'] * 30

# 转为字典
c_j = warehouse_data.set_index('仓库')['仓租月成本'].to_dict()
C_j = warehouse_data.set_index('仓库')['仓库上限 (月)'].to_dict()
P_j = warehouse_data.set_index('仓库')['产能上限 (月)'].to_dict()

# 计算每个品类的最高月销量
daily_sale_data['日期'] = pd.to_datetime(daily_sale_data['日期']) # 确保日期为日期格式
daily_sale_data['月份'] = daily_sale_data['日期'].dt.to_period('M') # 按月分组

# 按品类和月份求和，取每个品类最高月销量
monthly_sales = daily_sale_data.groupby(['品类', '月份'])['销售量'].sum().reset_index()
s_i = monthly_sales.groupby('品类')['销售量'].max().to_dict() # 每个品类的最大月销量

# 直接获取每个品类的月库存量最高值
p_i = monthly_store_data.groupby('品类')['预测库存量 1 (sarima)'].max().to_dict() # 每个品类的最大月库存量

# 处理件型和高级品类信息
category_info = category_data.set_index('品类')[['件型', '高级品类']]
category_info['高级品类'] = pd.to_numeric(category_info['高级品类'], errors='coerce').fillna(0)

# 处理关联度数据
R_ij = defaultdict(int, correlation_data.set_index(['品类 1', '品类 2'])['关联度'].to_dict())
for i in s_i.keys():
    for j in s_i.keys():
        if i != j and (i, j) not in R_ij:
            R_ij[(i, j)] = 0 # 默认关联度为 0

# 设置初始权重
w1, w2, w3, w4 = 10, 10, 50, 1 # 高级品类、件型、关联度、成本的权重
```

```

# 综合收益计算函数
def calculate_profit(solution):
    high_category_profit = 0
    uniformity_profit = 0
    association_profit = 0
    total_cost = 0

    # 记录每个仓库的分配数量
    warehouse_distribution = defaultdict(int)

    # 遍历所有仓库
    for warehouse in set(wh for whs in solution.values() for wh in whs):
        assigned_categories = [cat for cat, whs in solution.items() if warehouse in whs]

        # 检查高级品类和件型
        high_category_seen = set()
        type_seen = set()

        # 遍历分配到该仓库的品类
        for cat in assigned_categories:
            # 处理高级品类
            high_cat = category_info.loc[cat]['高级品类']
            if high_cat in high_category_seen:
                high_category_profit += 1000 # 如果发现重复的高级品类，增加收益
            else:
                high_category_seen.add(high_cat)

            # 处理件型
            type_cat = category_info.loc[cat]['件型']
            if type_cat in type_seen:
                uniformity_profit += 1000 # 如果发现重复的件型，增加收益
            else:
                type_seen.add(type_cat)

        # 更新每个仓库的分配数量
        warehouse_distribution[warehouse] += s_i[cat]

    # 高级品类收益（总和）计算
    high_category_profit += sum(category_info.loc[cat]['高级品类'] for cat in assigned_categories)

    # 关联度收益按比例计算
    for i in range(len(assigned_categories)):
        for j in range(i + 1, len(assigned_categories)):
            cat1, cat2 = assigned_categories[i], assigned_categories[j]
            association = R_ij[(cat1, cat2)]

            proportion = (
                (warehouse in solution[cat1]) / len(solution[cat1]) *
                (warehouse in solution[cat2]) / len(solution[cat2])
            )
            association_profit += association * proportion

    # 检查每个仓库的分配是否均匀
    for warehouse, total in warehouse_distribution.items():
        num_categories = len([cat for cat in solution if warehouse in solution[cat]])
        ideal_distribution = total / num_categories if num_categories > 0 else 0

        for cat in solution:
            if warehouse in solution[cat]:
                actual_distribution = s_i[cat]
                # 添加惩罚逻辑

```

```

        if actual_distribution < ideal_distribution * 0.9: # 假设小于 90%为惩罚
            total_cost += 100 # 设定惩罚成本

# 综合收益值计算
profit = -(w1 * high_category_profit + w2 * uniformity_profit + w3 * association_profit) + w4 * total_cost

# 输出各项收益
print("综合收益组成部分: ")
print(f"高级品类收益: {high_category_profit}")
print(f"件型一致性收益: {uniformity_profit}")
print(f"关联度收益: {association_profit}")
print(f"总成本: {total_cost}")

return profit

# 模拟退火算法
def simulated_annealing(categories, warehouses, max_iter=1000, initial_temp=1000, cooling_rate=0.99):
    current_solution = {
        i: set(random.sample(warehouses, random.randint(1, 3))) for i in categories
    }
    current_profit = calculate_profit(current_solution)

    best_solution = current_solution.copy()
    best_profit = current_profit

    temp = initial_temp

    for iteration in range(max_iter):
        # 生成新解
        new_solution = {i: current_solution[i].copy() for i in categories}
        category_to_modify = random.choice(categories)

        # 确保尽量均匀分配
        if len(new_solution[category_to_modify]) < 3 and random.random() < 0.5:
            new_solution[category_to_modify].add(random.choice(warehouses))
        elif len(new_solution[category_to_modify]) > 1:
            new_solution[category_to_modify].remove(random.choice(list(new_solution[category_to_modify])))

        new_profit = calculate_profit(new_solution)

        # 接受准则
        if new_profit < current_profit or random.random() < np.exp((current_profit - new_profit) / temp):
            current_solution = new_solution
            current_profit = new_profit

        if current_profit < best_profit:
            best_solution = current_solution.copy()
            best_profit = current_profit

        # 输出当前温度
        print(f"当前温度: {temp:.4f}")

        temp *= cooling_rate

    return best_solution, best_profit

# 执行模拟退火算法
categories = list(s_i.keys())
warehouses = list(c_j.keys())

```

```
# 设置退火参数
max_iter = 500 # 最大迭代次数
initial_temp = 1000 # 初始温度
cooling_rate = 0.99 # 温度衰减率

best_solution, best_profit = simulated_annealing(categories, warehouses, max_iter, initial_temp, cooling_rate)

# 输出结果
result = [(cat, list(best_solution[cat])) for cat in categories]
result_df = pd.DataFrame(result, columns=["Category", "Assigned_Warehouses"])

# 文件路径
output_file = "一品多仓分仓方案结果.xlsx"

# 检查文件是否存在，如果存在则删除
if os.path.exists(output_file):
    os.remove(output_file)

# 保存结果到 Excel，文件会被覆盖
result_df.to_excel(output_file, index=False)
print(f'最优解已输出到 '一品多仓分仓方案结果.xlsx'，综合收益值: {best_profit}')
```